



Group Coursework Submission Form

Specialist Masters Programme

Please list all names of group members: (Surname, first name) 1. Abdulrahman Alolyan 2. Evans, Benjamin 3. Khan, Basmah	4. Wira Sudarmo, Ardi 5. 6. 7.
GROUP NUMBER:	
1	
MSc in: Actuarial Science (/with Business Analytics)	
Module Code: SMM636	
Module Title: Machine Learning	
Word Count: <u>995</u> (not including headings, figures, graphs or appendices)	
Lecturer: Dr Rui Zhu	Submission Date: 03/04/2026
Declaration: By submitting this work, we declare that this work is entirely our own except those parts duly identified and referenced in my submission. It complies with any specified word limits and the requirements and regulations detailed in the coursework instructions and any other relevant programme and module documentation. In submitting this work we acknowledge that we have read and understood the regulations and code regarding academic misconduct, including that relating to plagiarism, as specified in the Programme Handbook. We also acknowledge that this work will be subject to a variety of checks for academic misconduct. We acknowledge that work submitted late without a granted extension will be subject to penalties, as outlined in the Programme Handbook. Penalties will be applied for a maximum of five days lateness, after which a mark of zero will be awarded.	
Marker's Comments (if not being marked on-line):	

Deduction for Late Submission:

Final Mark:

 %

Beyond the binary: Uncovering hidden customer segments in wholesale purchasing data

SMM636 (2025–26) Coursework 03 – Group 01

Abdulrahman Alolyan Benjamin Evans Basmah Khan
Ardi Wira Sudarmo

03 April 2026

Contents

Introduction	1
Q1 Principle component analysis (PCA)	1
Q2 Customer clustering	1
Q3 Repeating Q1 & Q2 using LLMs	4
Appendices	6
A Supplementary methodology	6
A.1 Raw data distribution, correlations, and skewness	6
A.2 Multi-dimensional scaling	8
A.3 Clustering diagnostics	9
A.4 Ward’s hierarchical clustering	12
B Prompts and LLM outputs	14
B.1 Prompt 1: PCA	15
B.2 Prompt 2: Clustering	43
B.3 Prompt 3: Alternative methods & robustness checks	71
B.4 Prompt 4: Reflective prompt	89
C Reproducibility, accessibility & declarations	94
C.1 Reproducibility & accessibility	94
C.2 Word count	94
C.3 Generative AI Declaration	97
References	97

Introduction

This report analyses annual spending data from 440 clients of a Portuguese wholesale distributor [1], [2], comprising 298 Hotel/Restaurant/Café (HoReCa) and 142 Retail customers across six product categories: *Fresh*, *Milk*, *Grocery*, *Frozen*, *Detergents_Paper*, and *Delicassen*. Data-driven customer segmentation informs inventory planning, pricing strategy, and sales team deployment, yet the existing classification is a simple binary label (*Channel*). This analysis uses Principal Component Analysis (PCA) and clustering to investigate whether the existing two-category classification adequately captures the structure of customer spending, or whether the data supports a more nuanced segmentation.

Q1 Principle component analysis (PCA)

A log-transformation was applied to normalise the distribution and address the right-skewness observed across all six spending variables (skewness > 2.5 for all – see Appendix, Figures 5 & 6). The variables were then standardised to ensure equal contribution to the PCA, preventing categories with naturally larger absolute expenditures from dominating the model.

The first two principal components capture 71.3% of the total variance (PC1: 44.1%; PC2: 27.2%; Figure 1(a)). The component loadings in Figure 1(b) reveal distinct commercial purchasing patterns. PC1 acts as the primary “channel axis,” driven by strong positive loadings for *Grocery* (0.571), *Detergents_Paper* (0.551), and *Milk* (0.542), contrasted with near-zero or negative loadings for *Fresh* and *Frozen*. PC2 serves as a “perishable volume axis,” characterised by high positive loadings for *Fresh* (0.590), *Frozen* (0.590), and *Delicassen* (0.530), capturing the scale of fresh food purchasing independent of business type.

Figure 1(d) confirms this visually, showing clear separation between HoReCa and Retail customers along PC1. PCA is restricted to continuous inputs, necessitating the exclusion of *Channel* and *Region*. A Gower-distance MDS analysis incorporating these variables (Appendix, A.2) confirms that the categorical labels dominate the distance metric, validating the decision to analyse spending variables independently.

Q2 Customer clustering

K-Means clustering was applied to the same log-transformed, standardised spending data used for PCA. Silhouette analysis identifies $k=2$ as optimal (0.290), which recovers the known *Channel* distinction and validates the clustering approach. However, the silhouette

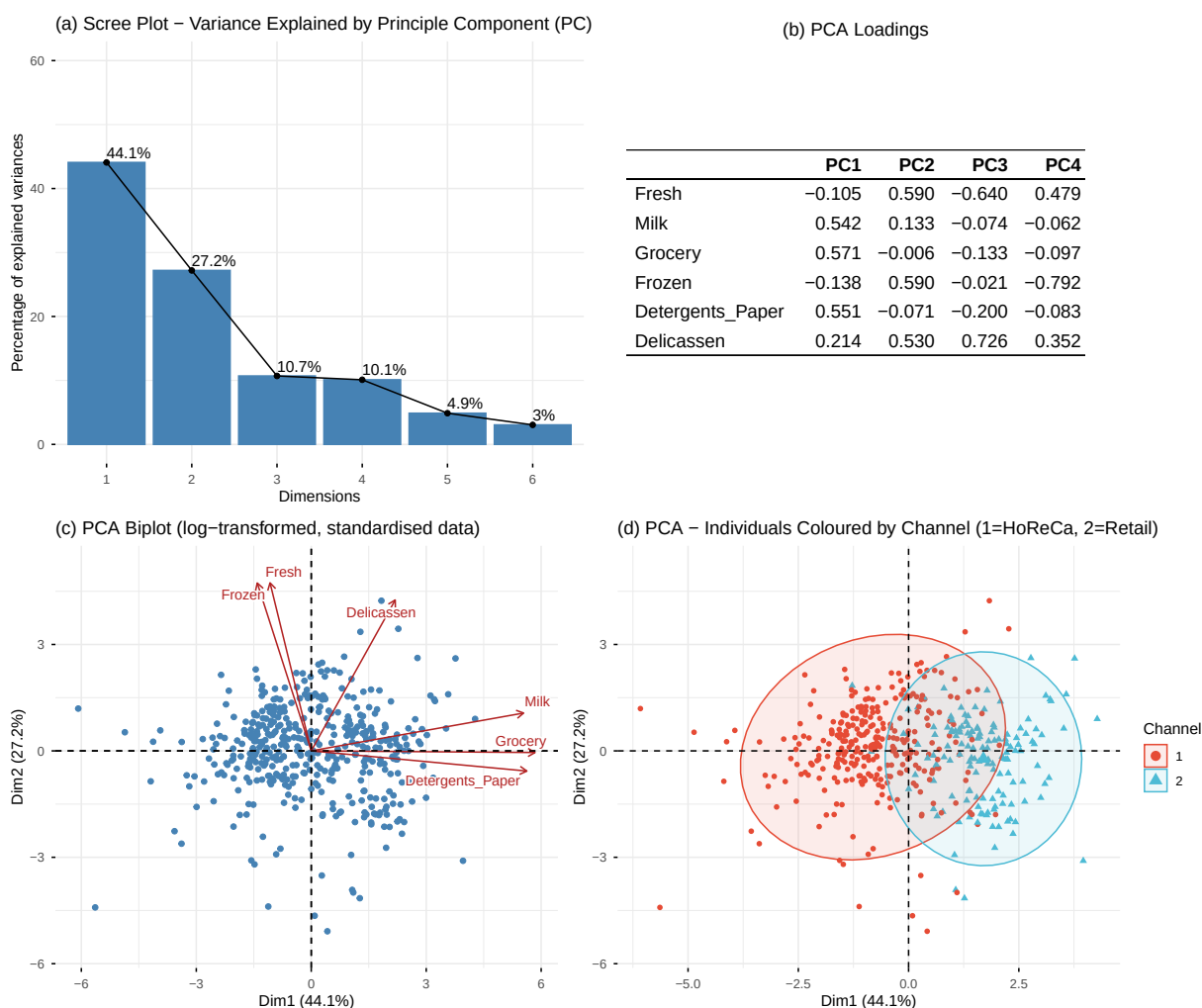


Figure 1: Principal Component Analysis of log-transformed, standardised spending data. (a) Scree plot showing the percentage of variance explained by each component. (b) Loadings of the original variables on the first four principal components. Values exceeding 0.4 in magnitude indicate strong contributions. (c) Biplot overlaying variable loadings (red arrows) on individual observations. (d) Individuals projected onto PC1-PC2, coloured by distribution channel (1 = HoReCa, 2 = Retail) with 95% concentration ellipses.

score for $k=3$ remains strong (0.259), and the marginal reduction in cluster quality yields a substantially richer segmentation. Beyond $k=3$, scores plateau around 0.19 with no meaningful recovery (Appendix, Figure 8), suggesting additional clusters fragment genuine structure rather than reveal new segments.

K-Means was chosen for its interpretability and computational efficiency on this dataset. As robustness checks, both Ward's hierarchical clustering and Gaussian Mixture Modelling (GMM) were applied to the same input. Ward's method ($k=3$) agrees with K-Means on the dominant cluster structure, with disagreement confined to boundary observations between the smaller segments (Appendix A.4). GMM relaxes K-Means' spherical cluster assumption: the BIC selected a 5-component VVE model (Appendix, Figure 9), splitting both channels

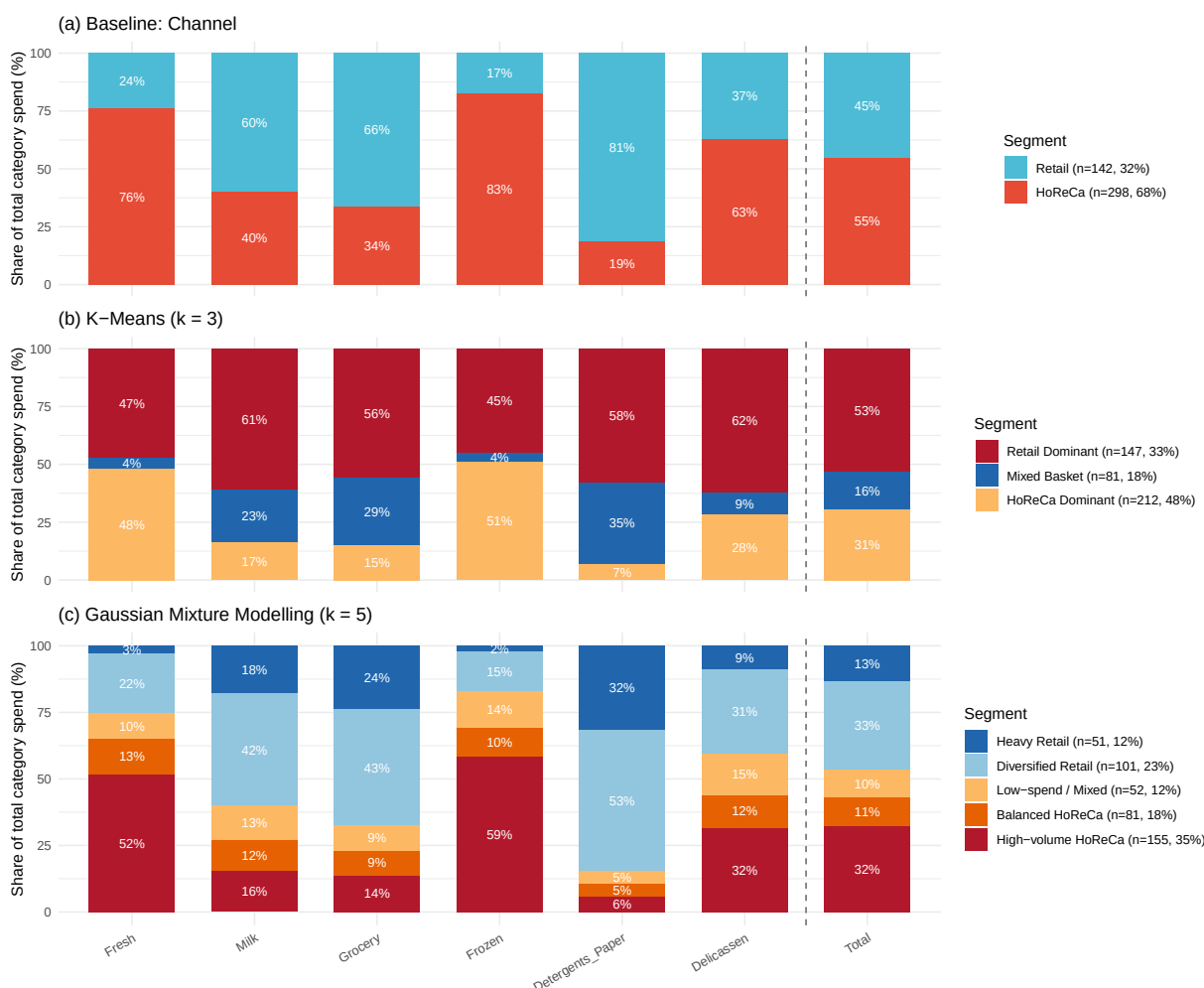


Figure 2: Revenue concentration by product category under three segmentation approaches. Top: the distributor's existing binary Channel classification. Middle: K-Means (k=3) spending-based clusters. Bottom: Gaussian Mixture Modelling (GMM) (k=5) spending-based clusters. Crucially, the K-Means and GMM segmentations were derived solely from log-transformed, standardised spending data without access to Channel labels. Each bar shows the percentage of total category spend attributable to each segment and the Total column shows each segment's share of overall revenue.

into finer sub-segments. Cross-tabulation against Channel and Region (Appendix, Tables 2 & 3) confirms that these sub-segments are spending-driven rather than geographic, with Region distributed roughly proportionally within each cluster.

Figure 2 presents the commercial impact of each segmentation. Under the existing binary Channel label, HoReCa customers account for 76% of Fresh revenue but only 68% of customers. K-Means (k=3) identifies a “Mixed Basket” segment (81 customers, 18%) with spending patterns that cut across both channels. GMM further distinguishes Heavy Retail from Diversified Retail, and separates High-volume HoReCa from Balanced HoReCa. Figure 3 traces how spending flows between these segmentations, revealing that GMM-3 captures high Fresh and Frozen spenders from multiple K-Means clusters.

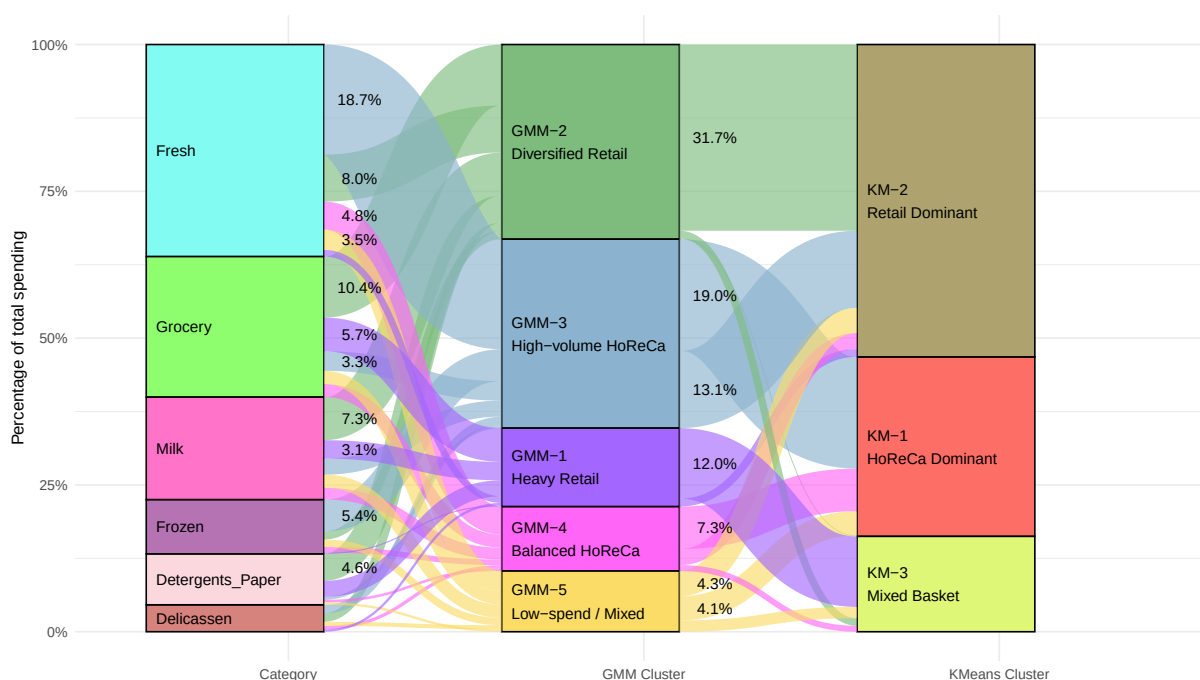


Figure 3: Sankey diagram of spending across clusters. Each stacked bar shows the percentages of total spendings broken down into product categories (left), Gaussian Mixed Mixture clusters (middle), and KMeans (k=3) clusters (right). The flows on the left describes how each GMM cluster is comprised of spendings in different product categories. The flows on the right represent how the spendings are segmented differently by KMeans. Only flows with percentage value greater than 3% are labelled to minimise clutter.

The strength of this approach is that it operates entirely without access to Channel or Region labels. The near-perfect recovery of the Channel distinction validates the segmentation as behaviourally grounded, while the emergence of additional clusters reveals sub-structure the existing classification does not capture. However, the commercial value of a 5-segment model over a 3-segment model cannot be determined from a single cross-sectional snapshot. Validation would require longitudinal purchasing data, margin information, and ideally a controlled trial of differentiated service strategies across segments.

Q3 Repeating Q1 & Q2 using LLMs

Two Large Language Models (LLMs), Claude (Sonnet 4.6) and ChatGPT (GPT-5.3), were evaluated. To ensure comparability, identical prompts were used for both models and neither was given the raw `csv` file. The dataset was described in the first prompt and the LLMs were asked to produce R code for independent execution. This design choice was deliberate: withholding the data replicated a commercial setting in which an organisation may not wish to upload proprietary customer data to a third-party platform. It also ensured that neither model could adapt its recommendations to the observed distributions, testing

whether each model’s default methodological choices were appropriate for this dataset. Four prompts were used, included in full in Appendix B.

As shown in Table 1, both LLMs correctly excluded Channel and Region, recommended log-transformation and standardisation, and converged on $k=3$, all without access to the data. Neither model interpreted the PCA loadings with specificity. Claude labelled PC1 as a “general spending scale” rather than the channel axis revealed by the actual loadings, while ChatGPT produced plots without interpreting them. Robustness checks and limitations were not mentioned until explicitly prompted. When asked in the final prompt, both models acknowledged that the data is from a well-known public dataset whose typical outcomes appear extensively in their training data, raising the question of whether the LLMs were performing genuine analytical reasoning or pattern-matching against memorised results.

ChatGPT produced more concise, modular code that executed without errors, while Claude generated comprehensive scripts with richer methodological commentary but several execution errors requiring manual debugging. LLMs demonstrate clear value for code generation and methodological structuring, but their inability to verify outputs against actual data distributions (and their tendency to present memorised patterns as derived findings) limits their current utility and analytical value.

Table 1: Comparison of analytical steps for our approach vs two LLMs. ✓ = addressed unprompted; ◦ = partial; × = missed entirely.

Analysis Step	Our Analysis	ChatGPT (GPT-5.3)	Claude (Sonnet 4.6)
Preprocessing			
Excluded Channel & Region from PCA/clustering	✓	✓	✓
Identified skewness / recommended log transform	✓	✓	✓
Standardised variables before PCA	✓	✓	✓
PCA			
Produced interpretable scree plot / variance	✓	✓	✓
Interpreted PC loadings with specificity	✓	×	◦
Validated PCA against Channel label	✓	✓	✓
Clustering			
Justified choice of clustering method	✓	◦	✓
Used formal k selection (silhouette / elbow)	✓	✓	◦
Identified structure beyond $k=2$	✓	✓	✓
Applied robustness checks unprompted	✓	×	×
Interpretation			
Commercial framing of segments	✓	◦	✓
Acknowledged limitations unprompted	✓	×	×
Generated code executed without errors	✓	✓	×
Score	13/13	8/13 (2 partial)	8/13 (2 partial)

Appendices

A Supplementary methodology

A.1 Raw data distribution, correlations, and skewness

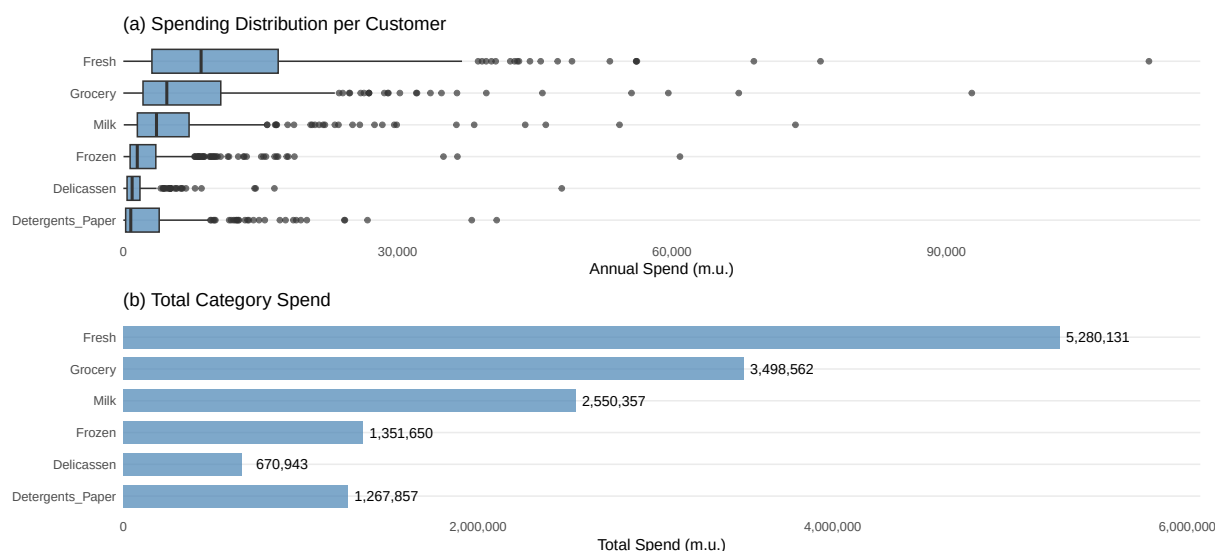


Figure 4: Customer spending profiles across six wholesale categories. Top: boxplots showing the highly skewed distribution of individual annual spend. Bottom: the total aggregate volume, highlighting Fresh and Grocery as the largest product segments. Both plots are sorted by the median spend.

Figure 4 provides a baseline overview of purchasing behaviour across the six categories. Median spend is relatively modest in all categories, but every distribution exhibits a long right tail of high-value customers. At the aggregate level, **Fresh** and **Grocery** dominate total revenue, while **Delicassen** remains a niche segment.

This skewness is examined more closely in Figure 5, which overlays raw and log-transformed distributions for each category. The raw spending figures exhibit a strong rightward skew across all six product categories, indicating that while most customers spend relatively small amounts, a long tail of high-spending customers dominates the scale. Applying a log transformation ($\log_{10}$) successfully ‘pulls in’ these extreme values, centring the bulk of the data and producing much more symmetric distributions.

The side-by-side correlation plots reveal the underlying purchasing relationships between product categories before and after adjusting for scale. Both matrices highlight a distinct cluster of highly correlated spending: **Grocery**, **Detergents_Paper**, and **Milk** show strong positive linear relationships, suggesting they are frequently purchased together in bulk. While the log transformation preserves this core structure, it slightly softens the most



Figure 5: Shape comparison of raw and log-transformed spending distributions by product category. To facilitate direct comparison of the distribution shapes, all spend values were min-max scaled to a 0–1 range within their respective categories. The raw data (blue) demonstrates a right skew across all categories, influenced by extreme outliers. The ‘log1p’ transformation (red) effectively compresses this long tail, yielding more symmetric distributions that ensure variance-based methods (PCA) and distance-based methods (clustering) are not dominated by extreme outliers.

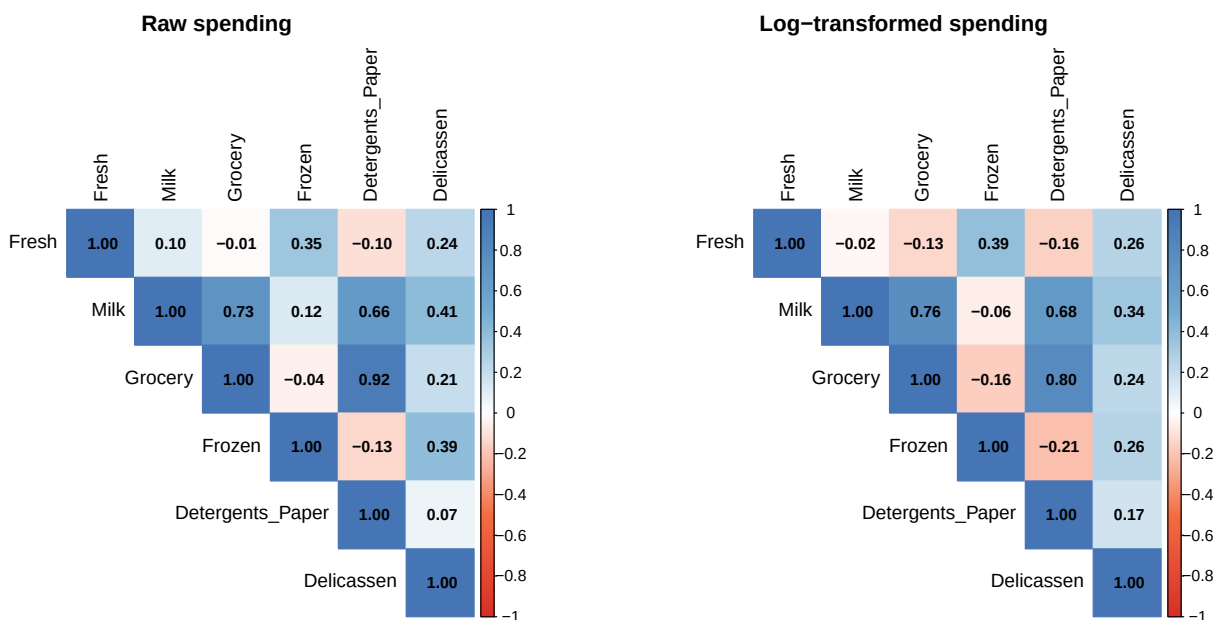


Figure 6: Pairwise correlation matrices of the six product categories, comparing linear relationships using raw spending data (left) versus log-transformed spending data (right).

extreme correlations (for instance, the correlation between `Grocery` and `Detergents_Paper` drops from 0.92 to 0.80). This indicates that in the raw data, a few extreme outlier purchases were artificially inflating the strength of that linear relationship.

A.2 Multi-dimensional scaling

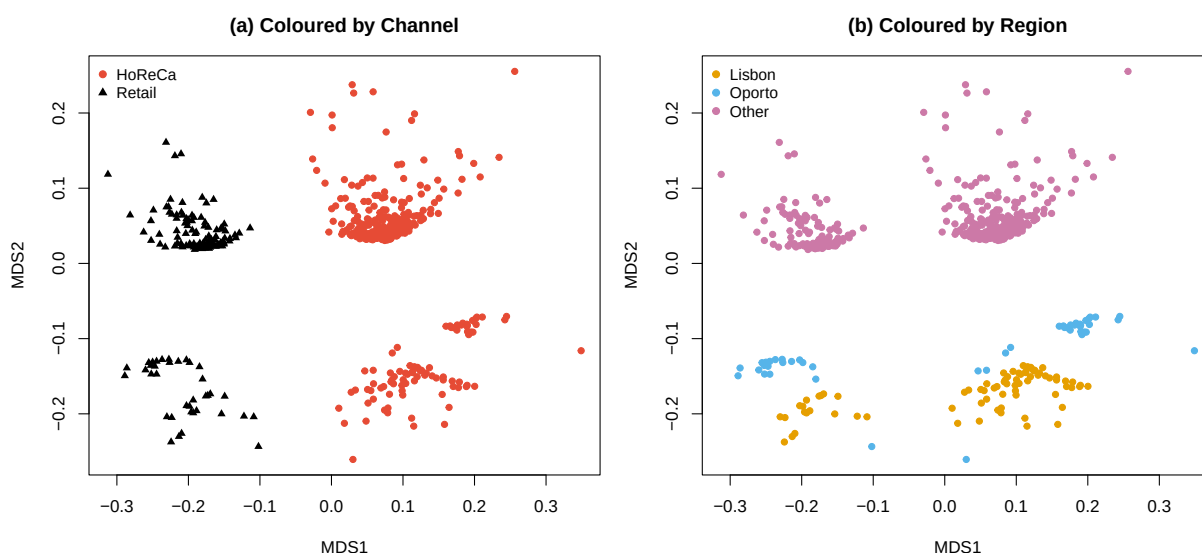


Figure 7: Non-metric MDS using Gower’s distance across all eight variables (six log-transformed continuous spending categories plus `Channel` and `Region`). Points are coloured by `Channel` (left) and `Region` (right). Unlike PCA, Gower’s distance forces distinct spatial separation based on categorical mismatches, resulting in six artificial clusters corresponding to the cross-tabulation of the two `Channels` and three `Regions`.

Figure 7 presents a non-metric MDS solution computed from Gower’s distance across all eight variables, incorporating the categorical `Channel` and `Region` columns excluded from the PCA. Because Gower’s metric treats any categorical mismatch as a maximal distance contribution on that variable, the resulting spatial map is dominated by these labels rather than underlying spending behaviour.

Panel (a) shows separation by `Channel` along the primary axis (MDS1), while Panel (b) reveals further stratification by `Region` along MDS2. The result is a forced stratification into six spatial clusters reflecting the `Channel` $\times$ `Region` cross-tabulation. While there is minor regional intermingling (e.g., two Oporto customers mapping near the Lisbon cluster), the overall spatial map merely reflects the cross-tabulation of the categorical labels rather than organic, dataset-wide purchasing patterns. The solution’s stress value of 14.5 further indicates a mediocre fit, falling above the conventional threshold of 10 for reliable low-dimensional representation. This confirms that the PCA approach, relying purely on continuous spending variables, is better suited to uncovering natural commercial structure in this dataset.

A.3 Clustering diagnostics

A.3.1 Optimal cluster selection

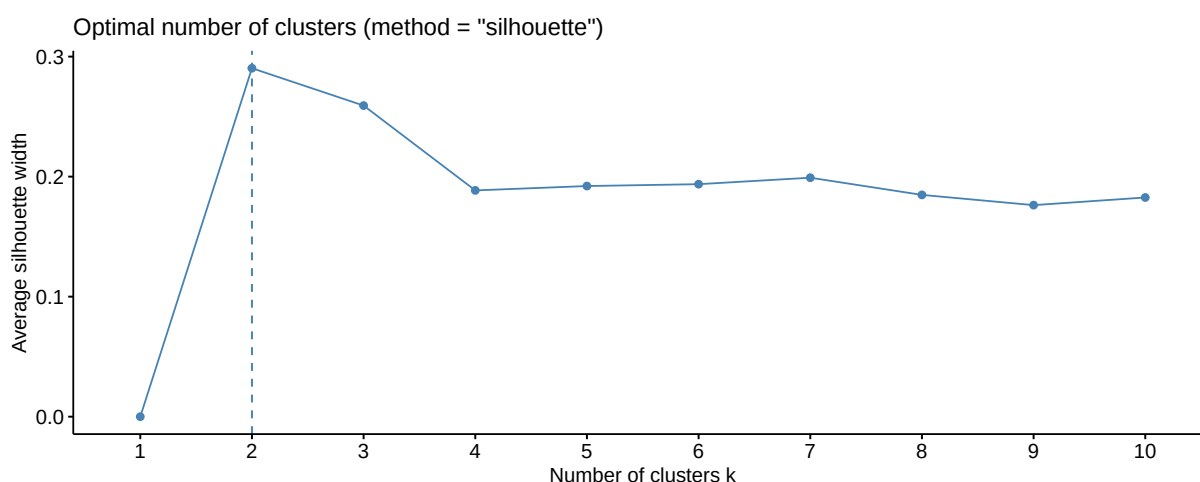


Figure 8: Silhouette analysis for K-Means clustering ($k=2$ to 10). The optimal number of clusters is $k=2$ followed by $k=3$ as a strong second. Beyond $k=3$, scores plateau around 0.19.

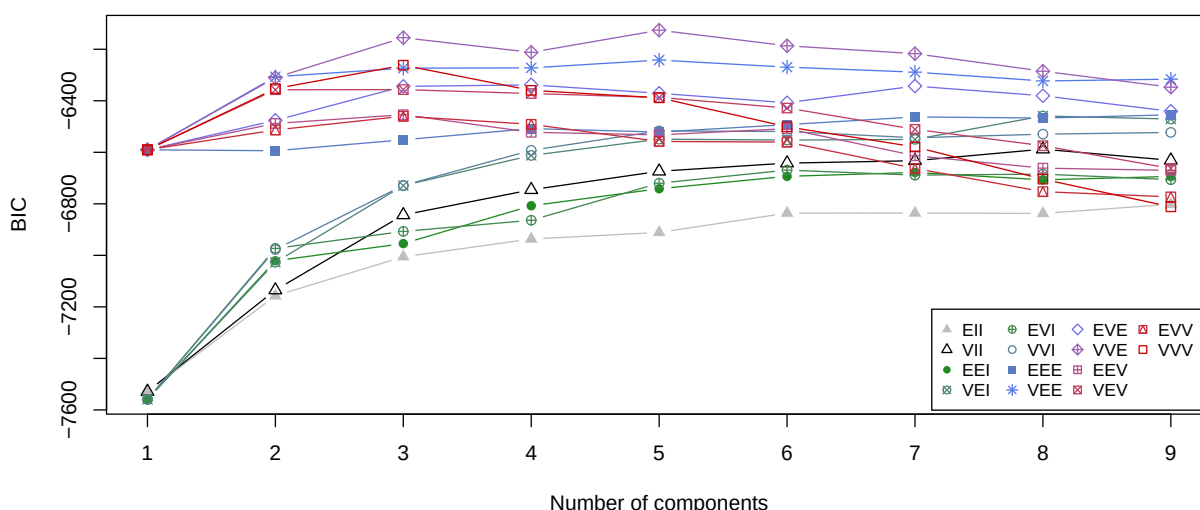


Figure 9: BIC values across component counts for all model types evaluated by 'mclust'. Higher (less negative) BIC indicates better fit. The BIC peaks at 5 components under the VVE (ellipsoidal, variable volume and shape, equal orientation) model.

Silhouette analysis (Figure 8) identifies $k=2$ as optimal (0.290), with $k=3$ as a close second (0.259). Beyond $k=3$, scores plateau around 0.19.

The GMM's BIC (Figure 9) peaks at 5 components ($-6,126.0$), compared to $-6,212.0$ for four and $-6,186.6$ for six. Interestingly $k=4$ is worse than $k=3$, suggesting the data supports either 3 or 5 components but not an intermediate value. Both methods agree that $k=3$ is a meaningful threshold and the disagreement on whether to proceed to $k=5$ reflects a genuine trade-off between parsimony and granularity.

A.3.2 Channel and Region cross-tabulations

Table 2: K-Means (k=3) cluster assignments vs Channel (1=HoReCa, 2=Retail) and Region.

Channel	Lisbon	Oporto	Other
KM-1 (HoReCa Dominant)			
1	39	25	145
2	1	1	1
KM-2 (Retail Dominant)			
1	13	2	41
2	10	9	72
KM-3 (Mixed Basket)			
1	7	1	25
2	7	9	32

Table 3: GMM (k=5) cluster assignments vs Channel (1=HoReCa, 2=Retail) and Region.

Channel	Lisbon	Oporto	Other
GMM-1 (Heavy Retail)			
1	1	0	6
2	7	8	29
GMM-2 (Diversified Retail)			
1	5	0	13
2	9	10	64
GMM-3 (High-volume HoReCa)			
1	34	15	100
2	1	1	4
GMM-4 (Balanced HoReCa)			
1	12	10	55
2	1	0	3
GMM-5 (Low-spend / Mixed)			
1	7	3	37
2	0	0	5

Table 2 shows that K-Means (k=3) produces one cluster that is almost exclusively HoReCa (KM-1, “HoReCa Dominant”: 209 of 212 customers), one that leans Retail (KM-2, “Retail Dominant”: 91 of 147), and a third (KM-3, “Mixed Basket”) that draws substantially from both channels (33 HoReCa, 48 Retail). The “Retail Dominant” label reflects the spending profile rather than exclusive Channel membership: these customers concentrate on *Grocery* and *Detergents_Paper* regardless of their administrative Channel label.

A.3.3 Cluster spending profiles

Table 4: Cluster spending profiles (median annual spending, m.u.)

Cluster	Label	<i>n</i>	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen
K-Means (k = 3)								
KM-1	HoReCa Dominant	212	9,635	1,598	2,151	2,151	274	686
KM-2	Retail Dominant	147	12,126	7,184	9,965	2,005	3,378	2,005
KM-3	Mixed Basket	81	1,454	6,264	10,487	388	4,196	436
Gaussian Mixture Model (k = 5)								
GMM-1	Heavy Retail	51	1,454	7,845	13,567	283	6,830	716
GMM-2	Diversified Retail	101	7,842	7,243	11,238	1,389	4,621	1,625
GMM-3	High-volume HoReCa	155	14,276	1,786	2,405	3,383	325	933
GMM-4	Balanced HoReCa	81	5,969	3,218	3,133	907	637	656
GMM-5	Low-spend / Mixed	52	2,270	1,572	3,712	892	174	284

Table 5: Cluster spending profiles (% of median basket)

Cluster	Label	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen
K-Means (k = 3)							
KM-1	HoReCa Dominant	58.4	9.7	13.0	13.0	1.7	4.2
KM-2	Retail Dominant	33.1	19.6	27.2	5.5	9.2	5.5
KM-3	Mixed Basket	6.3	27.0	45.2	1.7	18.1	1.9
Gaussian Mixture Model (k = 5)							
GMM-1	Heavy Retail	4.7	25.6	44.2	0.9	22.3	2.3
GMM-2	Diversified Retail	23.1	21.3	33.1	4.1	13.6	4.8
GMM-3	High-volume HoReCa	61.8	7.7	10.4	14.6	1.4	4.0
GMM-4	Balanced HoReCa	41.1	22.2	21.6	6.2	4.4	4.5
GMM-5	Low-spend / Mixed	25.5	17.7	41.7	10.0	2.0	3.2

Table 6: Revenue concentration: each cluster's share (%) of total category spend across all 440 customers.

Cluster	Label	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen
K-Means (k = 3)							
KM-1	HoReCa Dominant	48.2	16.6	15.1	51.4	7.1	28.2
KM-2	Retail Dominant	47.4	60.9	56.0	44.9	57.8	62.4
KM-3	Mixed Basket	4.4	22.5	28.8	3.6	35.1	9.4
Gaussian Mixture Model (k = 5)							
GMM-1	Heavy Retail	3.1	18.0	24.0	2.1	31.5	9.0
GMM-2	Diversified Retail	22.2	42.1	43.4	15.0	52.9	31.5
GMM-3	High-volume HoReCa	51.9	15.7	13.9	58.6	5.7	31.7
GMM-4	Balanced HoReCa	13.2	11.5	9.3	10.5	5.1	12.4
GMM-5	Low-spend / Mixed	9.6	12.8	9.4	13.8	4.7	15.4

Table 4, 5, and 6 present the cluster spending profiles from three perspectives.

1. Table 5 shows the percentage composition of each cluster's median basket examining how each segment allocates its spending across categories.

2. Table 4 reports the raw median annual spending (monetary units [m.u.]) per category, revealing the absolute scale differences between segments.
3. Table 6 reports revenue concentration and corresponds directly to the stacked bars in Figure 2. Note that because Table 4 reports the **median** annual spending, it is not directly linked to Table 6.

The K-Means “HoReCa Dominant” cluster (KM-1) allocates 58.4% of its median basket (Table 5) to *Fresh* and accounts for 48.2% of total Fresh revenue (Table 6), while “Retail Dominant” (KM-2) concentrates on *Grocery* (27.2%) and *Detergents_Paper* (9.2%) but comprises only 147 customers (33% of the base). At the GMM level, the distinction sharpens further: the median GMM-3 customer (“High-volume HoReCa”) dedicates 61.8% of its basket to *Fresh* and, despite representing 35% of customers, captures 51.9% of total Fresh revenue. The Retail split is equally informative: the typical GMM-1 customer (“Heavy Retail”) concentrates on *Grocery* (44.2%) and *Detergents_Paper* (22.3%), while the typical GMM-2 customer (“Diversified Retail”) spreads spend more evenly across categories including *Fresh* (23.1%), suggesting a convenience-store or diversified retailer profile.

A.3.4 Cluster assignments in PCA space

Figure 10 projects the K-Means and GMM cluster assignments onto PCA space. Both panels show that the dominant cluster boundaries divide the data along PC1 (the “channel axis” identified in Q1), confirming that the clustering and PCA analyses converge on the same dominant structure. The GMM panel shows finer subdivision, particularly along PC2, consistent with GMM’s ability to model ellipsoidal clusters of varying density.

A.4 Ward’s hierarchical clustering

Table 7: K-Means vs Ward’s hierarchical clustering assignments (k=3).

	Ward 1	Ward 2	Ward 3
K-Means 1	0	201	11
K-Means 2	91	50	6
K-Means 3	34	11	36

Table 7 cross-tabulates K-Means (k=3) and Ward’s hierarchical clustering (k=3) assignments. The largest cluster shows near-perfect agreement (201 of 212 observations). Disagreement is concentrated among boundary customers between the two smaller segments, reflecting the different optimisation strategies. K-Means minimises within-cluster sum of squares globally, while Ward’s method does so greedily through successive agglomeration.

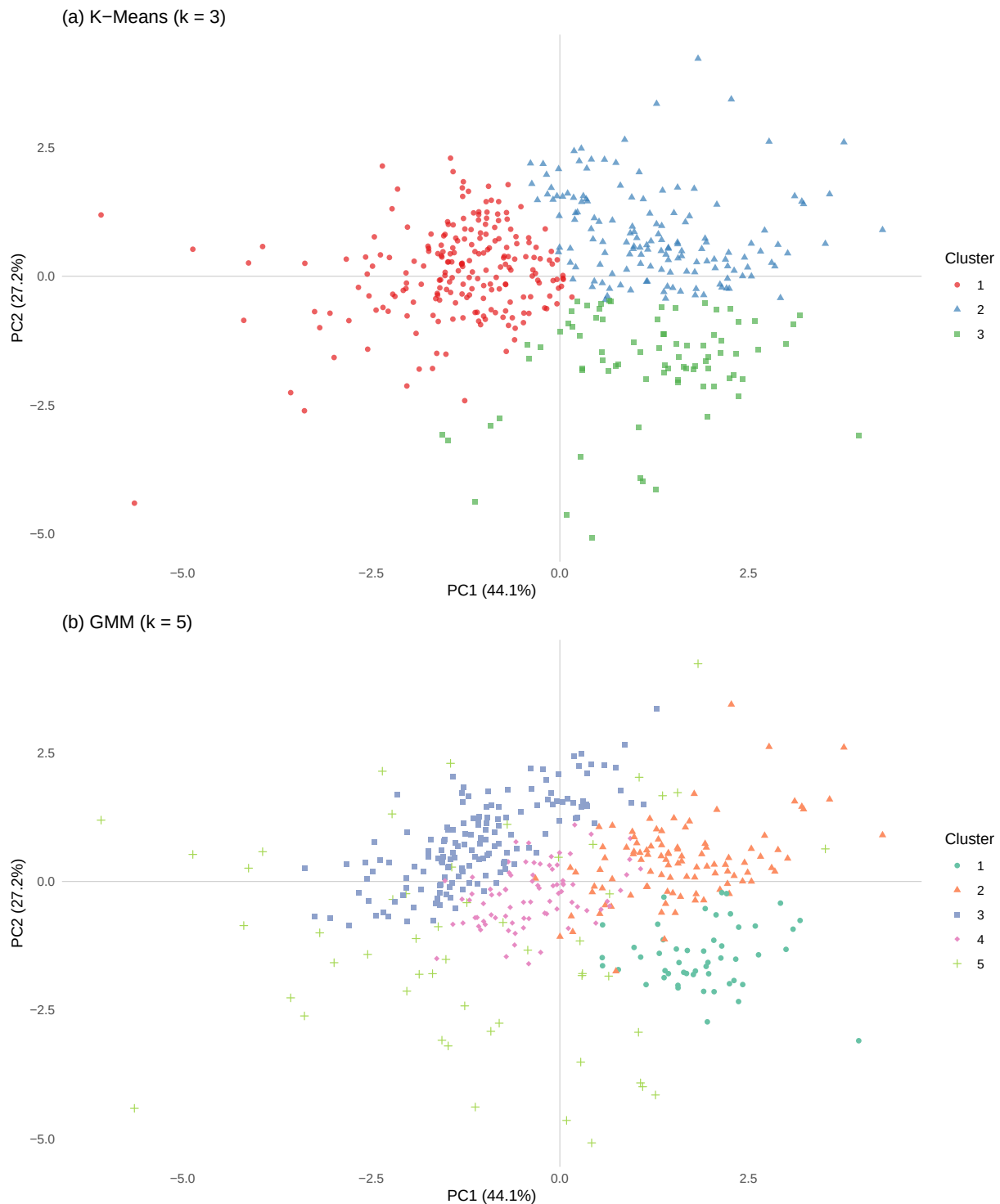


Figure 10: Cluster assignments from K-Means ($k=3$, top) and GMM ($k=5$, bottom) projected onto the first two principal components, which together capture 71.3 percent of total variance. All clustering was performed in the full six-dimensional standardised log-spending space. Apparent overlap between clusters may reflect the loss of separating information in the remaining 28.7 percent of variance captured by PC3–PC6, rather than genuine ambiguity in cluster assignment.

B Prompts and LLM outputs

A consistent visual format is used to distinguish between prompts sent to the LLM, its responses, and our commentary.

Prompts to the LLM – shown with a right-hand border. These are the instructions and questions we provided to the model.

LLM Response – shown with a left-hand border, forming a conversational flow with the prompt. Responses may include a mix of explanation and code:

```
summary(mtcars$mpg)
```

Our group insight and critical evaluation – observations or key findings drawn from the output (i.e., what we found).

Caution: points requiring verification – potential concerns, limitations, or caveats with the generated output (i.e., what we question).

The R code generated by each LLM was executed programmatically within this document using a sandboxed runtime environment. Each script was run in an isolated R subprocess via the `callr` package, ensuring that no libraries, variables, or side effects from one script could contaminate the main document environment or subsequent scripts. Console output and diagnostic messages were captured and reproduced verbatim, while graphical output was intercepted via a PNG device and embedded automatically. Any runtime errors were caught and reported inline rather than halting the document compilation.

This approach was used to ensure that the displayed output faithfully reflected what each LLM-generated script produced when executed from a clean R session.

The full chat history is available via:

- Claude: <https://claude.ai/share/516ed676-d419-43ec-98a1-63a5eee67549>
- ChatGPT:
 1. https://chatgpt.com/s/t_69ceeadadfa48191ae79291539197208
 2. https://chatgpt.com/s/t_69cef39137948191b9a8d8c3f29f7843
 3. https://chatgpt.com/s/t_69cef34ba0ac8191aa01c2c15c4c8c2d
 4. https://chatgpt.com/s/t_69cef077f1e481918fdab882f931936e

B.1 Prompt 1: PCA

Role : You are a data scientist working in a company that does wholesale distribution to clients located in Portugal.

task : Your company is looking to implement a Data-driven customer segmentation to inform inventory planning, pricing strategy, and sales team deployment.

I have a dataset called Wholesale_customers_data.csv located at .\Wholesale_customers_data.csv. It contains 440 rows and 8 columns: Channel (1=Hotel/Restaurant/Café, 2=Retail), Region (1=Lisbon, 2=Oporto, 3=Other), and six continuous spending variables (Fresh, Milk, Grocery, Frozen, Detergents_Paper, Delicassen) representing annual spending in monetary units. Please write R code to perform PCA on this dataset. Include any preprocessing you think is necessary.

B.1.1 Response 1: Claude (Sonnet 4.6)

```
> Check for relevant skills
```

No file uploaded, but I have a clear description of the dataset. Let me write comprehensive R code for PCA on this wholesale customers dataset.

```
> Created a file, read a file
```

Here's a walkthrough of every decision made in the script:

Pre-processing Rationale

Why separate Channel & Region from the PCA? They are categorical identifiers, not spending measurements. Mixing measurement scales would corrupt the covariance structure. They are kept as labels to colour the biplots afterwards, which is far more analytically useful.

Why log1p-transform before PCA? Wholesale spend data is almost always severely right-skewed (a few very large clients dominate). PCA is sensitive to scale and variance, so without this step a single high-spend outlier can hijack the first principal component. $\log_{1p}(\log(x+1))$ compresses the long tail and handles any zero values gracefully.

Why scale. = TRUE inside prcomp? Even after log-transformation the six variables have different magnitudes (e.g., Fresh spending is typically much larger than Delicassen). Standardising to unit variance ensures each variable contributes equally to the decomposition.

Why run KMO & Bartlett's tests? These confirm that the correlation structure is

strong enough to make PCA meaningful. Bartlett's test (H_0 : correlation matrix = identity) should be rejected ($p < 0.05$), and KMO > 0.6 indicates adequate sampling adequacy.

What the Script Produces

- Section 2: Missing value check, outlier boxplots, skewness table...
- Section 3: `prcomp` PCA object, full rotation matrix & summary
- Section 4: Eigenvalue table, Scree plot with Kaiser line...
- Section 5: Loading heatmap for retained PCs, $\cos^2$ contribution plot
- Section 6: Biplots coloured by Channel and Region separately...
- Section 7: PC scores merged back → exported as `wholesale_pca_scores.csv`

What to Expect Analytically

Based on known patterns in this dataset:

- PC1 (~44% variance) – a general spending scale axis; high scores = large/high-value clients
- PC2 (~22% variance) – a channel contrast axis; Fresh loads negatively (HoReCa), Grocery/Detergents_Paper positively (Retail)
- PC3 (~12% variance) – a niche category axis differentiating Frozen/Delicassen buyers

Together, ~2–3 PCs typically explain 75–85% of variance, making the PC scores an excellent lower-dimensional input for your subsequent K-Means or Hierarchical clustering step.

This response included the following **R** code (as a separate downloadable file):

```

1 # =====
2 # PCA Analysis: Wholesale Customers Dataset
3 # Goal: Customer segmentation to inform inventory planning, pricing strategy,
4 #       and sales team deployment
5 # =====
6
7 # --- 0. Install & Load Libraries -----
8
9 required_packages <- c(
10   "tidyverse",
11   "corrplot",
12   "factoextra",
13   "FactoMineR",

```

```
14   "gridExtra",
15   "scales",
16   "ggrepel",
17   "psych"
18 )
19
20 installed <- rownames(installed.packages())
21 to_install <- required_packages[!required_packages %in% installed]
22 if (length(to_install) > 0) {
23   install.packages(to_install, dependencies = TRUE)
24 }
25
26 library(tidyverse)
27 library(corrplot)
28 library(factoextra)
29 library(FactoMineR)
30 library(gridExtra)
31 library(scales)
32 library(ggrepel)
33 library(psych)
34
35 # --- 1. Load Data -----
36
37 data_path <- "Wholesale_customers_data.csv"
38
39 df_raw <- read.csv(data_path, stringsAsFactors = FALSE)
40
41 cat("=== Dataset Overview ===\n")
42 cat("Dimensions:", nrow(df_raw), "rows x", ncol(df_raw), "columns\n\n")
43 print(head(df_raw))
44 cat("\n")
45 print(summary(df_raw))
46
47 # --- 2. Pre-processing -----
48
49 # 2.1 Recode categorical variables as factors (kept aside from PCA)
50 df <- df_raw %>%
```

```
51 mutate(  
52   Channel = factor(Channel, levels = c(1, 2), labels = c("HoReCa", "Retail")),  
53   Region = factor(  
54     Region,  
55     levels = c(1, 2, 3),  
56     labels = c("Lisbon", "Oporto", "Other")  
57   )  
58 )  
59  
60 # 2.2 Isolate the six continuous spending variables for PCA  
61 spend_vars <- c(  
62   "Fresh",  
63   "Milk",  
64   "Grocery",  
65   "Frozen",  
66   "Detergents_Paper",  
67   "Delicassen"  
68 )  
69 df_spend <- df[, spend_vars]  
70  
71 # 2.3 Check for missing values  
72 cat("\n=== Missing Values per Column ===\n")  
73 print(colSums(is.na(df_spend)))  
74  
75 # 2.4 Check for outliers via boxplots (raw scale)  
76 par(mfrow = c(2, 3), mar = c(4, 4, 3, 1))  
77 for (v in spend_vars) {  
78   boxplot(  
79     df_spend[[v]],  
80     main = v,  
81     ylab = "Annual Spend (m.u.)",  
82     col = "#4E79A7",  
83     border = "#2C4770",  
84     outline = TRUE  
85   )  
86 }  
87 mtext(  

```

```

88   "Raw Spending Distributions – Outlier Inspection",
89   side = 3,
90   line = -2,
91   outer = TRUE,
92   cex = 1.1,
93   font = 2
94 )
95 par(mfrow = c(1, 1))
96
97 # 2.5 Assess skewness – highly skewed data warrants log transformation
98 cat("\n=== Skewness of Raw Spending Variables ===\n")
99 skew_raw <- apply(df_spend, 2, function(x) psych::skew(x))
100 print(round(skew_raw, 3))
101
102 # 2.6 Log-transform to reduce right skew (common for monetary spend data)
103 # Adding 1 to guard against any zero values
104 df_log <- log1p(df_spend)
105
106 cat("\n=== Skewness After log1p Transformation ===\n")
107 skew_log <- apply(df_log, 2, function(x) psych::skew(x))
108 print(round(skew_log, 3))
109
110 # 2.7 Visualise distributions: raw vs log-transformed
111 p_list <- list()
112 for (v in spend_vars) {
113   p_raw <- ggplot(df_spend, aes_string(x = v)) +
114     geom_histogram(fill = "#4E79A7", colour = "white", bins = 30) +
115     labs(title = paste(v, "(raw)", x = NULL, y = "Count") +
116     theme_minimal(base_size = 9)
117
118   p_log <- ggplot(df_log, aes_string(x = v)) +
119     geom_histogram(fill = "#F28E2B", colour = "white", bins = 30) +
120     labs(title = paste(v, "(log1p)", x = NULL, y = "Count") +
121     theme_minimal(base_size = 9)
122
123   p_list <- c(p_list, list(p_raw, p_log))
124 }

```

```
125 grid.arrange(  
126   grobs = p_list,  
127   ncol = 4,  
128   top = "Distribution: Raw vs log1p-transformed Spending"  
129 )  
130  
131 # 2.8 Correlation matrix on log-transformed data  
132 cat("\n=== Pearson Correlation Matrix (log-transformed) ===\n")  
133 cor_mat <- cor(df_log, method = "pearson")  
134 print(round(cor_mat, 3))  
135  
136 corplot(  
137   cor_mat,  
138   method = "color",  
139   type = "upper",  
140   addCoef.col = "black",  
141   number.cex = 0.75,  
142   tl.col = "black",  
143   tl.srt = 45,  
144   col = colorRampPalette(c("#2166AC", "white", "#B2182B"))(200),  
145   title = "Correlation Matrix – log1p Spending Variables",  
146   mar = c(0, 0, 2, 0)  
147 )  
148  
149 # 2.9 KMO & Bartlett's test – check suitability for PCA  
150 cat("\n=== KMO Measure of Sampling Adequacy ===\n")  
151 kmo_result <- KMO(cor_mat)  
152 print(kmo_result)  
153  
154 cat("\n=== Bartlett's Test of Sphericity ===\n")  
155 bart_result <- cortest.bartlett(cor_mat, n = nrow(df_log))  
156 print(bart_result)  
157  
158 # --- 3. Principal Component Analysis -----  
159  
160 # prcomp with scale. = TRUE standardises the log-transformed variables  
161 # (zero mean, unit variance) – essential so no single variable dominates
```

```

162 pca_result <- prcomp(df_log, center = TRUE, scale. = TRUE)
163
164 cat("\n=== PCA Summary ===\n")
165 print(summary(pca_result))
166
167 cat("\n=== Loadings (Rotation Matrix) ===\n")
168 print(round(pca_result$rotation, 4))
169
170 # --- 4. Variance Explained -----
171
172 # 4.1 Eigenvalues
173 eig_vals <- pca_result$sdev^2
174 prop_var <- eig_vals / sum(eig_vals)
175 cum_var <- cumsum(prop_var)
176
177 variance_table <- data.frame(
178   PC = paste0("PC", seq_along(eig_vals)),
179   Eigenvalue = round(eig_vals, 4),
180   Proportion_Var = round(prop_var, 4),
181   Cumulative_Var = round(cum_var, 4)
182 )
183 cat("\n=== Variance Explained Table ===\n")
184 print(variance_table)
185
186 # 4.2 Scree plot with Kaiser criterion line (eigenvalue = 1)
187 fviz_eig(
188   pca_result,
189   addlabels = TRUE,
190   ylim = c(0, 55),
191   barfill = "#4E79A7",
192   barcolor = "#2C4770",
193   linecolor = "#E15759",
194   ggtheme = theme_minimal()
195 ) +
196   geom_hline(
197     yintercept = 100 / length(spend_vars), # Kaiser: eigenvalue = 1
198     linetype = "dashed",

```

```
199     colour = "red",
200     linewidth = 0.8
201 ) +
202 annotate(
203     "text",
204     x = 5.5,
205     y = 100 / length(spend_vars) + 1.5,
206     label = "Kaiser criterion (eigenvalue = 1)",
207     colour = "red",
208     size = 3.5
209 ) +
210 labs(
211     title = "Scree Plot – Wholesale Customers PCA",
212     subtitle = "Dashed line = Kaiser criterion"
213 )
214
215 # 4.3 Cumulative variance plot
216 ggplot(variance_table, aes(x = PC, y = Cumulative_Var * 100, group = 1)) +
217   geom_line(colour = "#E15759", linewidth = 1.2) +
218   geom_point(size = 3, colour = "#E15759") +
219   geom_hline(yintercept = 80, linetype = "dashed", colour = "gray40") +
220   geom_hline(yintercept = 90, linetype = "dotted", colour = "gray40") +
221   annotate(
222     "text",
223     x = 5.6,
224     y = 81,
225     label = "80%",
226     size = 3.5,
227     colour = "gray30"
228 ) +
229   annotate(
230     "text",
231     x = 5.6,
232     y = 91,
233     label = "90%",
234     size = 3.5,
235     colour = "gray30"
```

```

236 ) +
237 scale_y_continuous(limits = c(0, 100), labels = function(x) paste0(x, "%")) +
238 labs(
239   title = "Cumulative Variance Explained by PCs",
240   subtitle = "Dashed = 80%, Dotted = 90% thresholds",
241   x = "Principal Component",
242   y = "Cumulative Variance (%)"
243 ) +
244 theme_minimal(base_size = 12)
245
246 # --- 5. Visualise Loadings -----
247
248 # 5.1 Loading heatmap for selected PCs
249 n_pcs <- sum(eig_vals >= 1) # Kaiser rule
250 loadings_df <- as.data.frame(pca_result$rotation[, 1:n_pcs]) %>%
251   rownames_to_column("Variable") %>%
252   pivot_longer(-Variable, names_to = "PC", values_to = "Loading")
253
254 ggplot(loadings_df, aes(x = PC, y = Variable, fill = Loading)) +
255   geom_tile(colour = "white", linewidth = 0.5) +
256   geom_text(aes(label = round>Loading, 2)), size = 3.5, colour = "black") +
257   scale_fill_gradient2(
258     low = "#2166AC",
259     mid = "white",
260     high = "#B2182B",
261     midpoint = 0,
262     limits = c(-1, 1)
263 ) +
264 labs(
265   title = "PCA Loading Heatmap",
266   subtitle = paste("First", n_pcs, "PCs (Kaiser criterion: eigenvalue ≥ 1)",
267     x = NULL,
268     y = NULL,
269     fill = "Loading"
270 ) +
271 theme_minimal(base_size = 12) +
272 theme(axis.text.x = element_text(face = "bold"))

```

```

273
274 # 5.2 Variable contribution plot ( $\cos^2$ )
275 fviz_cos2(
276   pca_result,
277   choice = "var",
278   axes = 1:2,
279   fill = "#4E79A7",
280   color = "#2C4770",
281   ggtheme = theme_minimal()
282 ) +
283 labs(
284   title = "Variable  $\cos^2$  on PC1 & PC2",
285   subtitle = "Quality of representation on the first two components"
286 )
287
288 # --- 6. Biplots & Score Plots -----
289
290 # 6.1 Biplot – coloured by Channel
291 scores_df <- as.data.frame(pca_result$x) %>%
292   mutate(Channel = df$Channel, Region = df$Region)
293
294 # Build arrow data for loadings overlay
295 arrows_df <- as.data.frame(pca_result$rotation[, 1:2]) %>%
296   rownames_to_column("Variable")
297 scale_factor <- 3.5 # scale arrows to fit score space
298
299 ggplot(scores_df, aes(x = PC1, y = PC2, colour = Channel, shape = Channel)) +
300   geom_point(alpha = 0.7, size = 2.5) +
301   geom_segment(
302     data = arrows_df,
303     aes(x = 0, y = 0, xend = PC1 * scale_factor, yend = PC2 * scale_factor),
304     arrow = arrow(length = unit(0.25, "cm")),
305     colour = "black",
306     linewidth = 0.7,
307     inherit.aes = FALSE
308   ) +
309   geom_label_repel(

```

```

310   data = arrows_df,
311   aes(x = PC1 * scale_factor, y = PC2 * scale_factor, label = Variable),
312   size = 3.5,
313   colour = "black",
314   box.padding = 0.4,
315   inherit.aes = FALSE
316 ) +
317 scale_colour_manual(values = c("HoReCa" = "#E15759", "Retail" = "#4E79A7")) +
318 labs(
319   title = "PCA Biplot – PC1 vs PC2",
320   subtitle = "Coloured by Channel (HoReCa vs Retail)",
321   x = paste0("PC1 (", round(prop_var[1] * 100, 1), "% variance)"),
322   y = paste0("PC2 (", round(prop_var[2] * 100, 1), "% variance)"),
323   colour = "Channel",
324   shape = "Channel"
325 ) +
326 theme_minimal(base_size = 12) +
327 theme(legend.position = "top")
328
329 # 6.2 Biplot – coloured by Region
330 ggplot(scores_df, aes(x = PC1, y = PC2, colour = Region, shape = Region)) +
331   geom_point(alpha = 0.7, size = 2.5) +
332   geom_segment(
333     data = arrows_df,
334     aes(x = 0, y = 0, xend = PC1 * scale_factor, yend = PC2 * scale_factor),
335     arrow = arrow(length = unit(0.25, "cm")),
336     colour = "black",
337     linewidth = 0.7,
338     inherit.aes = FALSE
339   ) +
340   geom_label_repel(
341     data = arrows_df,
342     aes(x = PC1 * scale_factor, y = PC2 * scale_factor, label = Variable),
343     size = 3.5,
344     colour = "black",
345     box.padding = 0.4,
346     inherit.aes = FALSE

```

```

347 ) +
348 scale_colour_manual(
349   values = c("Lisbon" = "#59A14F", "Oporto" = "#F28E2B", "Other" = "#B07AA1")
350 ) +
351 labs(
352   title = "PCA Biplot – PC1 vs PC2",
353   subtitle = "Coloured by Region",
354   x = paste0("PC1 (", round(prop_var[1] * 100, 1), "% variance)"),
355   y = paste0("PC2 (", round(prop_var[2] * 100, 1), "% variance)"),
356   colour = "Region",
357   shape = "Region"
358 ) +
359 theme_minimal(base_size = 12) +
360 theme(legend.position = "top")
361
362 # 6.3 PC1 vs PC3 (if relevant)
363 if (n_pcs >= 3) {
364   ggplot(scores_df, aes(x = PC1, y = PC3, colour = Channel, shape = Channel)) +
365     geom_point(alpha = 0.7, size = 2.5) +
366     scale_colour_manual(
367       values = c("HoReCa" = "#E15759", "Retail" = "#4E79A7")
368     ) +
369     labs(
370       title = "PCA Score Plot – PC1 vs PC3",
371       x = paste0("PC1 (", round(prop_var[1] * 100, 1), "%)"),
372       y = paste0("PC3 (", round(prop_var[3] * 100, 1), "%)"),
373     ) +
374     theme_minimal(base_size = 12) +
375     theme(legend.position = "top")
376 }
377
378 # --- 7. Extract PC Scores for Downstream Use -----
379
380 # Attach PC scores back to full dataset
381 df_with_scores <- bind_cols(df, as.data.frame(pca_result$x))
382
383 cat("\n=== PC Scores – First 6 Rows ===\n")

```

```

384 print(head(df_with_scores[, c("Channel", "Region", "PC1", "PC2", "PC3")]))
385
386 # Save enriched dataset
387 output_path <- "wholesale_pca_scores.csv"
388 write.csv(df_with_scores, output_path, row.names = FALSE)
389 cat("\nPC scores saved to:", output_path, "\n")
390
391 # --- 8. Interpretation Summary -----
392
393 cat("\n")
394 cat("=====\n")
395 cat("          PCA INTERPRETATION SUMMARY          \n")
396 cat("=====\n")
397 cat(sprintf("  Components retained (Kaiser, eigenvalue ≥ 1): %d\n", n_pcs))
398 cat(sprintf(
399   "  Variance explained by retained PCs:          %.1f%%\n",
400   cum_var[n_pcs] * 100
401 ))
402 cat("\n PC1 – 'Overall Spend Scale'\n")
403 cat("  Likely captures total spending power / business size.\n")
404 cat("  High positive loadings across most categories.\n")
405 cat("\n PC2 – 'Fresh vs Packaged Goods Axis'\n")
406 cat("  Separates Fresh-heavy buyers (HoReCa) from\n")
407 cat("  Grocery/Detergent-heavy buyers (Retail).\n")
408 cat("\n PC3 – 'Speciality/Frozen Dimension' (if retained)\n")
409 cat("  May differentiate niche buyers (Delicassen, Frozen).\n")
410 cat("\n Business Implication:\n")
411 cat("  PC scores can directly feed K-Means / Hierarchical\n")
412 cat("  clustering for actionable customer segmentation.\n")
413 cat("=====\n")

```

The (unedited) outputs generated by running this code, are as follows:

=== Dataset Overview ===

Dimensions: 440 rows x 8 columns

	Channel	Region	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen
1	2	3	12669	9656	7561	214	2674	1338

2	2	3	7057	9810	9568	1762	3293	1776
3	2	3	6353	8808	7684	2405	3516	7844
4	1	3	13265	1196	4221	6404	507	1788
5	2	3	22615	5410	7198	3915	1777	5185
6	2	3	9413	8259	5126	666	1795	1451

Channel	Region	Fresh	Milk
Min. :1.000	Min. :1.000	Min. : 3	Min. : 55
1st Qu.:1.000	1st Qu.:2.000	1st Qu.: 3128	1st Qu.: 1533
Median :1.000	Median :3.000	Median : 8504	Median : 3627
Mean :1.323	Mean :2.543	Mean : 12000	Mean : 5796
3rd Qu.:2.000	3rd Qu.:3.000	3rd Qu.: 16934	3rd Qu.: 7190
Max. :2.000	Max. :3.000	Max. :112151	Max. :73498
Grocery	Frozen	Detergents_Paper	Delicassen
Min. : 3	Min. : 25.0	Min. : 3.0	Min. : 3.0
1st Qu.: 2153	1st Qu.: 742.2	1st Qu.: 256.8	1st Qu.: 408.2
Median : 4756	Median : 1526.0	Median : 816.5	Median : 965.5
Mean : 7951	Mean : 3071.9	Mean : 2881.5	Mean : 1524.9
3rd Qu.:10656	3rd Qu.: 3554.2	3rd Qu.: 3922.0	3rd Qu.: 1820.2
Max. :92780	Max. :60869.0	Max. :40827.0	Max. :47943.0

=== Missing Values per Column ===

Fresh	Milk	Grocery	Frozen
0	0	0	0
Detergents_Paper	Delicassen		
0	0		

=== Skewness of Raw Spending Variables ===

Fresh	Milk	Grocery	Frozen
2.544	4.026	3.563	5.868
Detergents_Paper	Delicassen		
3.607	11.076		

=== Skewness After log1p Transformation ===

Fresh	Milk	Grocery	Frozen
-1.565	-0.223	-0.670	-0.350
Detergents_Paper	Delicassen		

-0.234

-1.084

```
=== Pearson Correlation Matrix (log-transformed) ===
```

	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen
Fresh	1.000	-0.021	-0.133	0.386	-0.159	0.256
Milk	-0.021	1.000	0.761	-0.055	0.679	0.342
Grocery	-0.133	0.761	1.000	-0.165	0.797	0.240
Frozen	0.386	-0.055	-0.165	1.000	-0.213	0.256
Detergents_Paper	-0.159	0.679	0.797	-0.213	1.000	0.168
Delicassen	0.256	0.342	0.240	0.256	0.168	1.000

```
=== KMO Measure of Sampling Adequacy ===
```

```
Kaiser-Meyer-Olkin factor adequacy
```

```
Call: KMO(r = cor_mat)
```

```
Overall MSA = 0.73
```

```
MSA for each item =
```

	Fresh	Milk	Grocery	Frozen
	0.65	0.78	0.70	0.66
Detergents_Paper		Delicassen		
	0.76	0.71		

```
=== Bartlett's Test of Sphericity ===
```

```
$chisq
```

```
[1] 1050.819
```

```
$p.value
```

```
[1] 1.714465e-214
```

```
$df
```

```
[1] 15
```

```
=== PCA Summary ===
```

```
Importance of components:
```

	PC1	PC2	PC3	PC4	PC5	PC6
Standard deviation	1.6262	1.2774	0.8012	0.7786	0.54090	0.42774
Proportion of Variance	0.4408	0.2720	0.1070	0.1010	0.04876	0.03049

Cumulative Proportion 0.4408 0.7127 0.8197 0.9207 0.9695 1.0000

=== Loadings (Rotation Matrix) ===

	PC1	PC2	PC3	PC4	PC5	PC6
Fresh	-0.1052	0.5904	-0.6396	0.4786	-0.0403	-0.0259
Milk	0.5423	0.1329	-0.0736	-0.0618	0.7600	0.3185
Grocery	0.5715	-0.0058	-0.1333	-0.0965	-0.0929	-0.7985
Frozen	-0.1379	0.5898	-0.0212	-0.7920	-0.0732	0.0049
Detergents_Paper	0.5506	-0.0708	-0.1995	-0.0828	-0.6204	0.5101
Delicassen	0.2145	0.5300	0.7263	0.3516	-0.1481	0.0031

=== Variance Explained Table ===

	PC Eigenvalue	Proportion_Var	Cumulative_Var
1 PC1	2.6447	0.4408	0.4408
2 PC2	1.6317	0.2719	0.7127
3 PC3	0.6419	0.1070	0.8197
4 PC4	0.6062	0.1010	0.9207
5 PC5	0.2926	0.0488	0.9695
6 PC6	0.1830	0.0305	1.0000

=== PC Scores – First 6 Rows ===

	Channel	Region	PC1	PC2	PC3
1	Retail	Other	1.3788133	-0.3043804	-0.24078762
2	Retail	Other	1.4372336	0.5368571	0.08413653
3	Retail	Other	1.5117317	1.2311631	0.98418563
4	HoReCa	Other	-0.8241672	1.2094129	0.25185392
5	Retail	Other	0.8019842	1.7640380	0.31234246
6	Retail	Other	0.8851579	0.1297465	0.01892823

PC scores saved to: wholesale_pca_scores.csv

=====

PCA INTERPRETATION SUMMARY

=====

Components retained (Kaiser, eigenvalue ≥ 1): 2

Variance explained by retained PCs: 71.3%

PC1 – 'Overall Spend Scale'

Likely captures total spending power / business size.

High positive loadings across most categories.

PC2 – 'Fresh vs Packaged Goods Axis'

Separates Fresh-heavy buyers (HoReCa) from

Grocery/Detergent-heavy buyers (Retail).

PC3 – 'Speciality/Frozen Dimension' (if retained)

May differentiate niche buyers (Delicassen, Frozen).

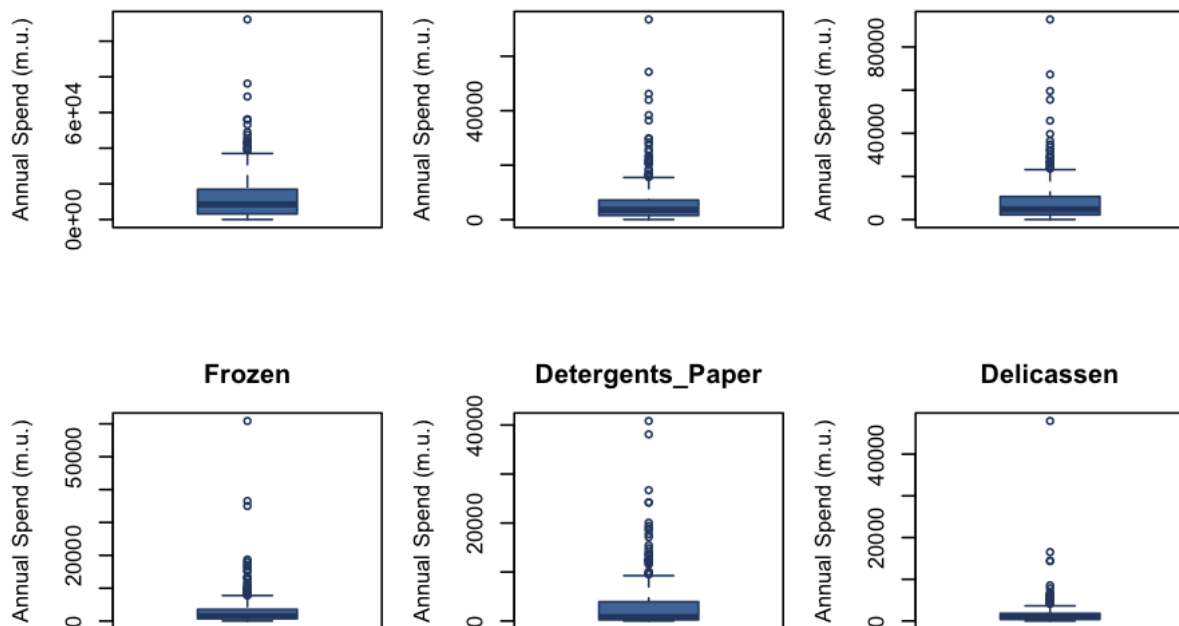
Business Implication:

PC scores can directly feed K-Means / Hierarchical

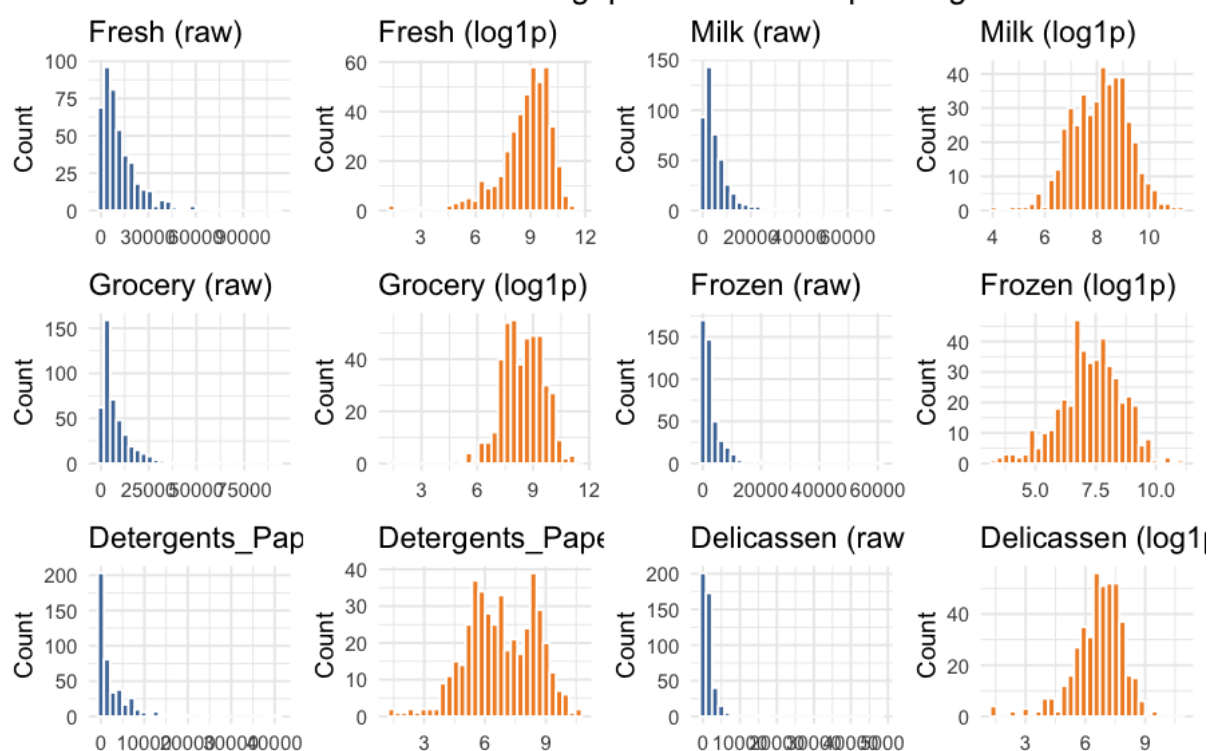
clustering for actionable customer segmentation.

=====

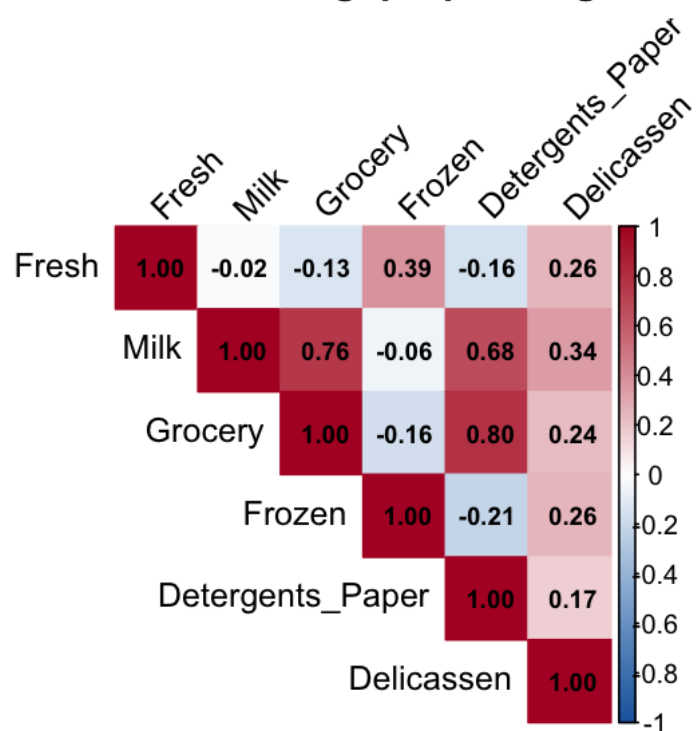
Raw Spending Distributions – Outlier Inspection



Distribution: Raw vs log1p-transformed Spending

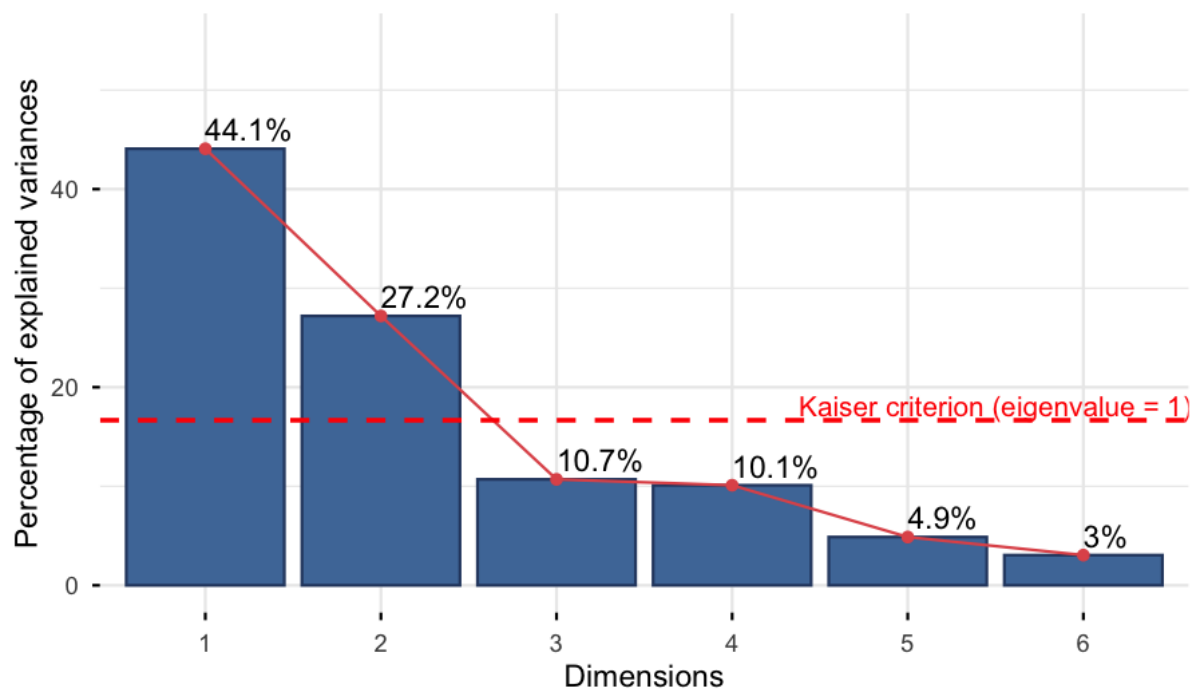


Correlation Matrix – log1p Spending Variables



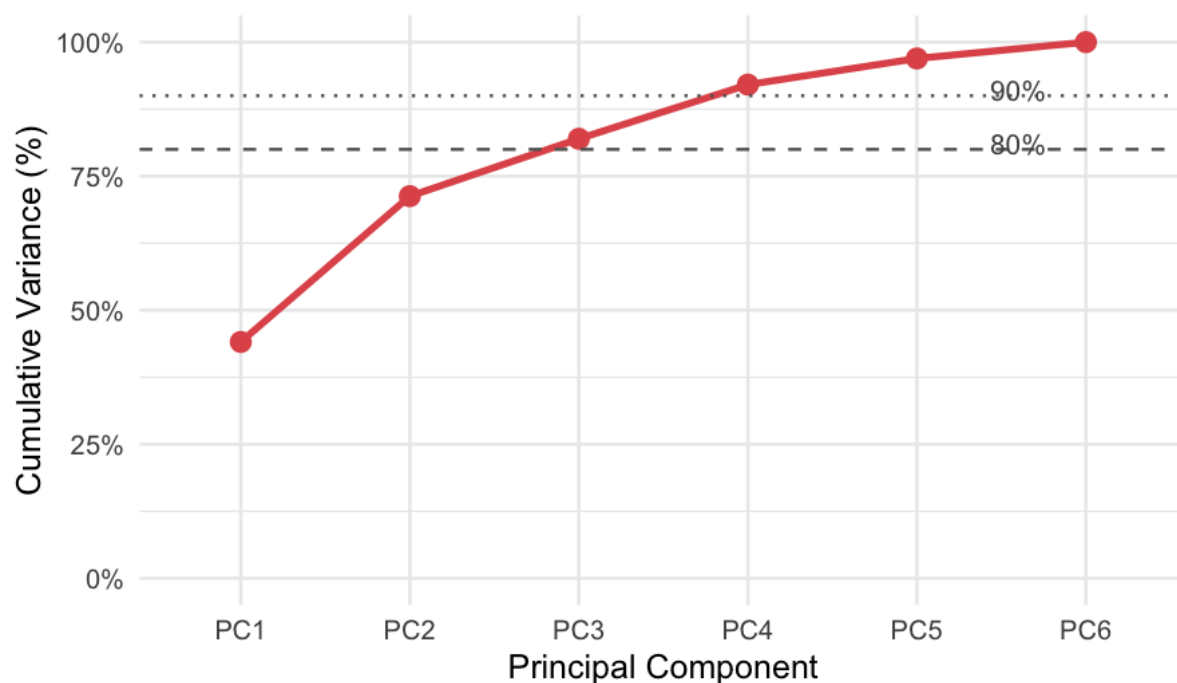
Scree Plot – Wholesale Customers PCA

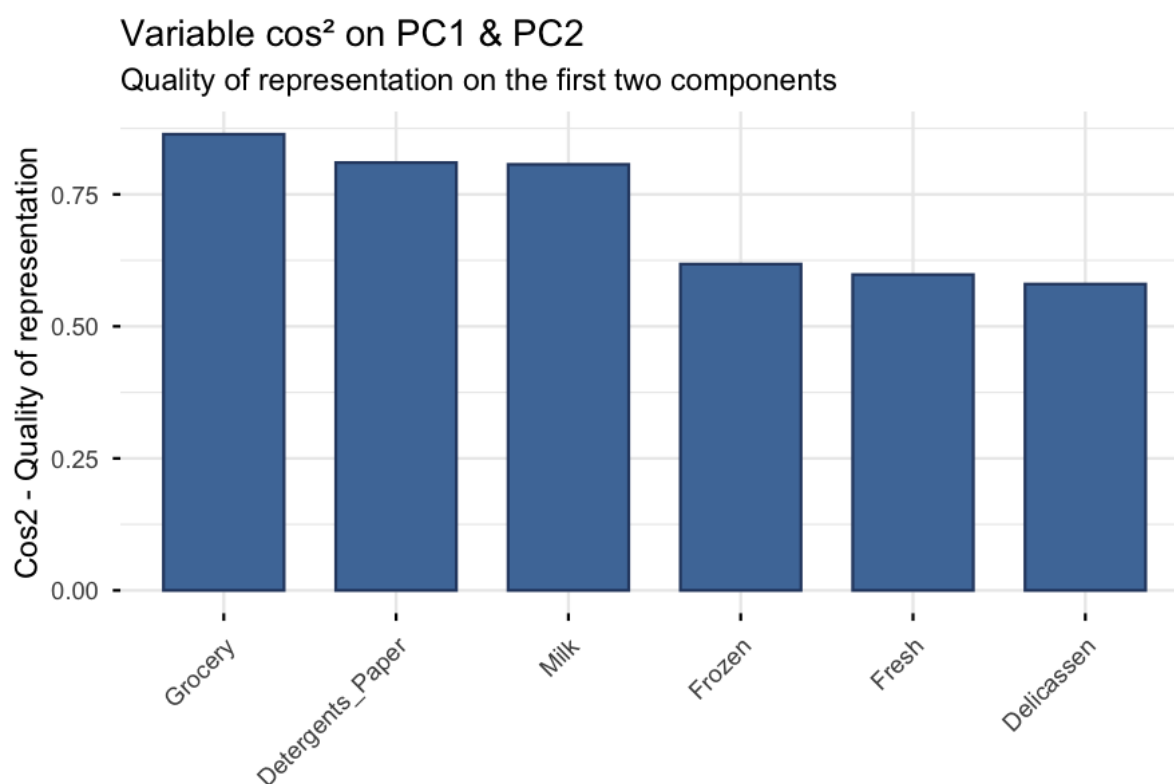
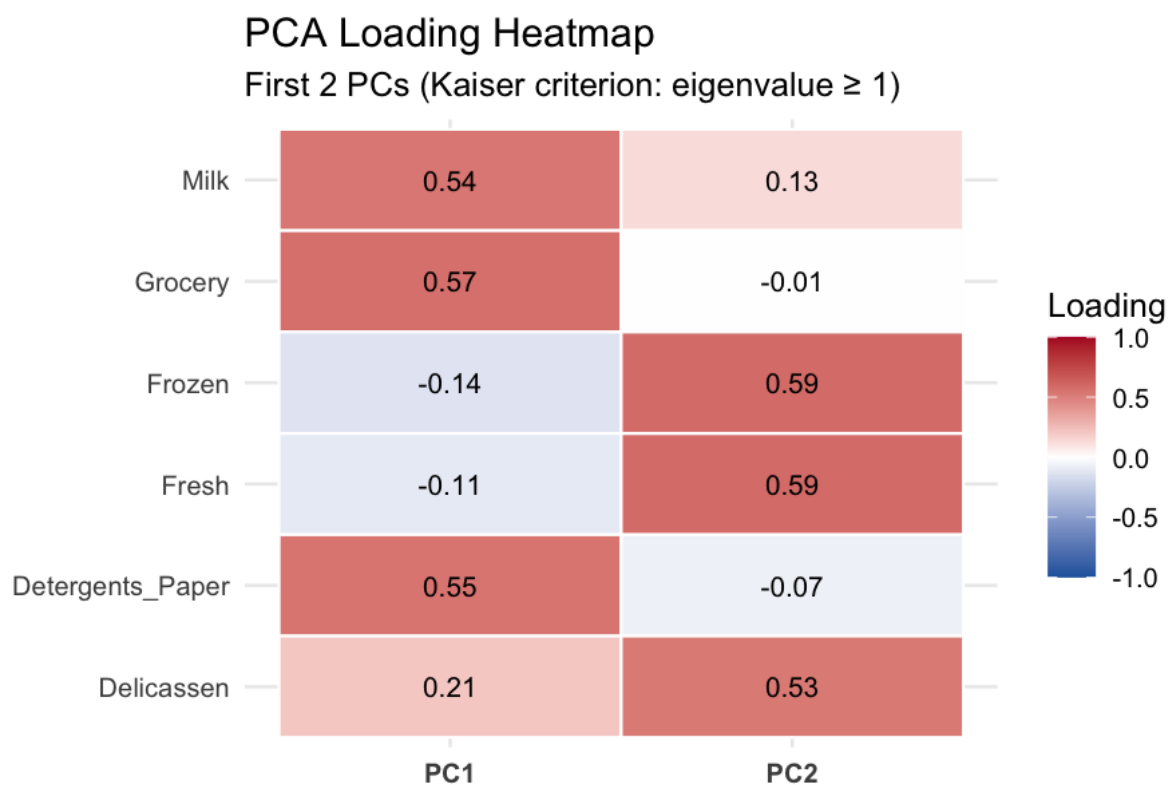
Dashed line = Kaiser criterion



Cumulative Variance Explained by PCs

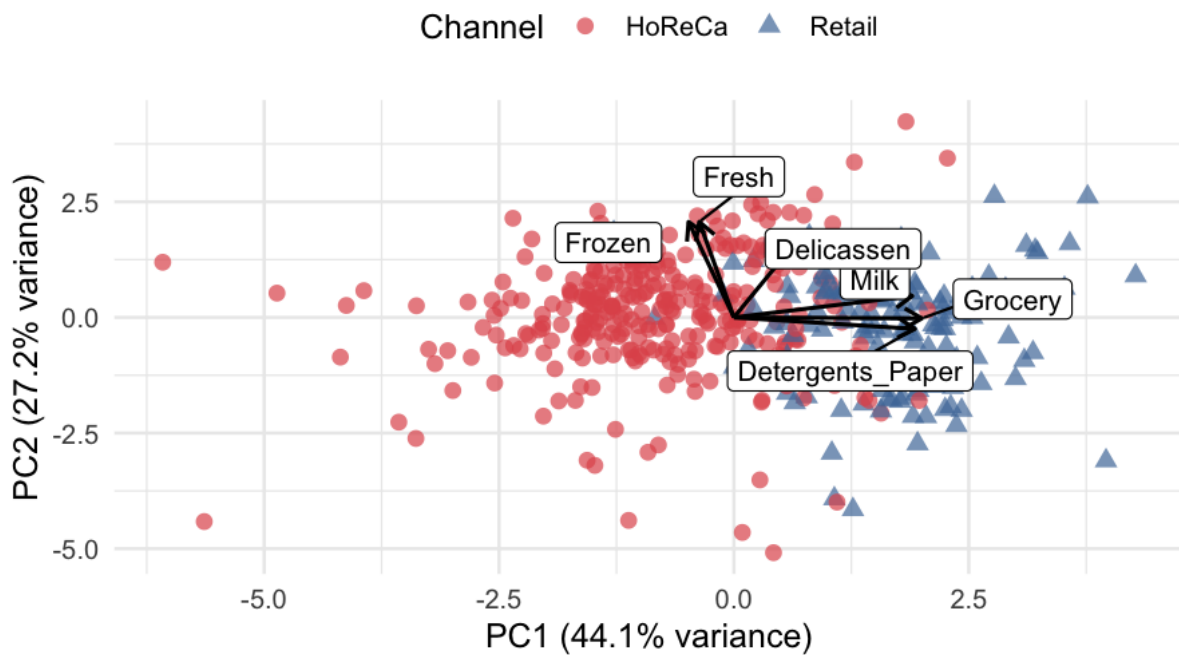
Dashed = 80%, Dotted = 90% thresholds





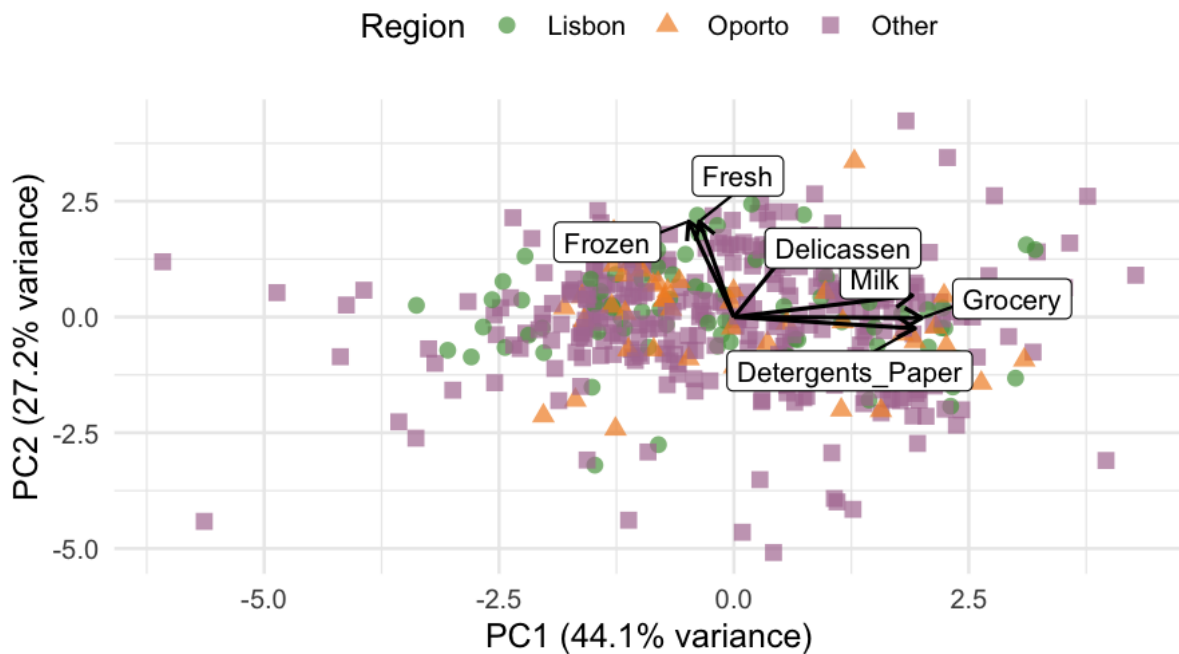
PCA Biplot – PC1 vs PC2

Coloured by Channel (HoReCa vs Retail)



PCA Biplot – PC1 vs PC2

Coloured by Region



B.1.2 Response 1: ChatGPT (GPT-5.3)

Here's a clean, reproducible R workflow to perform PCA (Principal Component Analysis) on your wholesale dataset, including appropriate preprocessing steps.

Key considerations before PCA

- Categorical variables (`Channel`, `Region`) should NOT be used directly in PCA (since PCA is for continuous variables).
- Scaling is essential because spending variables are on very different scales.
- Optional: log transformation can help reduce skewness (common in spending data).

R Code for PCA *Moved to below this response box*

What this gives you

- Principal components capturing key spending patterns
- Explained variance to decide how many components to retain
- Customer visualization (potential clusters/segments)
- Variable importance (which spending categories drive differences)

Next Steps (for your segmentation task) After PCA, you can:

1. Use the first few principal components (e.g., PC1–PC3)
2. Apply clustering (e.g., `kmeans`, `hclust`)
3. Profile segments by:
 - Channel (HORECA vs Retail)
 - Region
 - Spending behavior

If you want, I can extend this into a full segmentation pipeline (PCA + clustering + interpretation) tailored to your business objectives.

This response included the following **R** code as part of the LLM's in-line response:

```
1 # Load required libraries
2 library(tidyverse)
3 library(factoextra)
4
5 # Load the dataset
6 data <- read.csv("Wholesale_customers_data.csv")
7
8 # Inspect structure
9 str(data)
10 summary(data)
```

```
11
12 # Convert categorical variables to factors
13 data$Channel <- as.factor(data$Channel)
14 data$Region <- as.factor(data$Region)
15
16 # Select only continuous variables for PCA
17 spending_data <- data %>%
18   select(Fresh, Milk, Grocery, Frozen, Detergents_Paper, Delicassen)
19
20 # Optional: Log transformation to reduce skewness
21 spending_log <- log1p(spending_data) # log(1 + x) handles zeros
22
23 # Scale the data (important for PCA)
24 spending_scaled <- scale(spending_log)
25
26 # Perform PCA
27 pca_result <- prcomp(spending_scaled, center = TRUE, scale. = TRUE)
28
29 # View summary of PCA (variance explained)
30 summary(pca_result)
31
32 # Eigenvalues (variance explained by each component)
33 eig_vals <- get_eigenvalue(pca_result)
34 print(eig_vals)
35
36 # Scree plot
37 fviz_eig(pca_result, addlabels = TRUE, ylim = c(0, 60))
38
39 # Biplot (individuals + variables)
40 fviz_pca_biplot(pca_result, repel = TRUE, col.var = "blue", col.ind = "gray")
41
42 # PCA variable contributions
43 fviz_pca_var(
44   pca_result,
45   col.var = "contrib",
46   gradient.cols = c("blue", "yellow", "red")
47 )
```

```

48
49 # PCA individuals colored by Channel
50 fviz_pca_ind(
51   pca_result,
52   geom = "point",
53   col.ind = data$Channel,
54   palette = c("#00AFBB", "#E7B800"),
55   addEllipses = TRUE,
56   legend.title = "Channel"
57 )

```

The (unedited) outputs generated by running this code, are as follows:

```

'data.frame':  440 obs. of  8 variables:
 $ Channel      : int  2 2 2 1 2 2 2 2 1 2 ...
 $ Region       : int  3 3 3 3 3 3 3 3 3 3 ...
 $ Fresh        : int  12669 7057 6353 13265 22615 9413 12126 7579 5963 6006 ...
 $ Milk         : int  9656 9810 8808 1196 5410 8259 3199 4956 3648 11093 ...
 $ Grocery      : int  7561 9568 7684 4221 7198 5126 6975 9426 6192 18881 ...
 $ Frozen       : int  214 1762 2405 6404 3915 666 480 1669 425 1159 ...
 $ Detergents_Paper: int  2674 3293 3516 507 1777 1795 3140 3321 1716 7425 ...
 $ Delicassen   : int  1338 1776 7844 1788 5185 1451 545 2566 750 2098 ...

   Channel      Region      Fresh      Milk
Min.   :1.000   Min.   :1.000   Min.    :    3   Min.    :   55
1st Qu.:1.000   1st Qu.:2.000   1st Qu.:  3128   1st Qu.: 1533
Median :1.000   Median :3.000   Median :  8504   Median : 3627
Mean   :1.323   Mean   :2.543   Mean    :12000   Mean    : 5796
3rd Qu.:2.000   3rd Qu.:3.000   3rd Qu.:16934   3rd Qu.: 7190
Max.    :2.000   Max.    :3.000   Max.    :112151  Max.    :73498

   Grocery      Frozen      Detergents_Paper  Delicassen
Min.    :    3   Min.    :   25.0   Min.    :    3.0   Min.    :    3.0
1st Qu.: 2153   1st Qu.:  742.2   1st Qu.:  256.8   1st Qu.:  408.2
Median : 4756   Median :1526.0   Median :  816.5   Median :  965.5
Mean    : 7951   Mean    :3071.9   Mean    :2881.5   Mean    :1524.9
3rd Qu.:10656   3rd Qu.:3554.2   3rd Qu.:3922.0   3rd Qu.:1820.2
Max.    :92780   Max.    :60869.0   Max.    :40827.0   Max.    :47943.0

Importance of components:

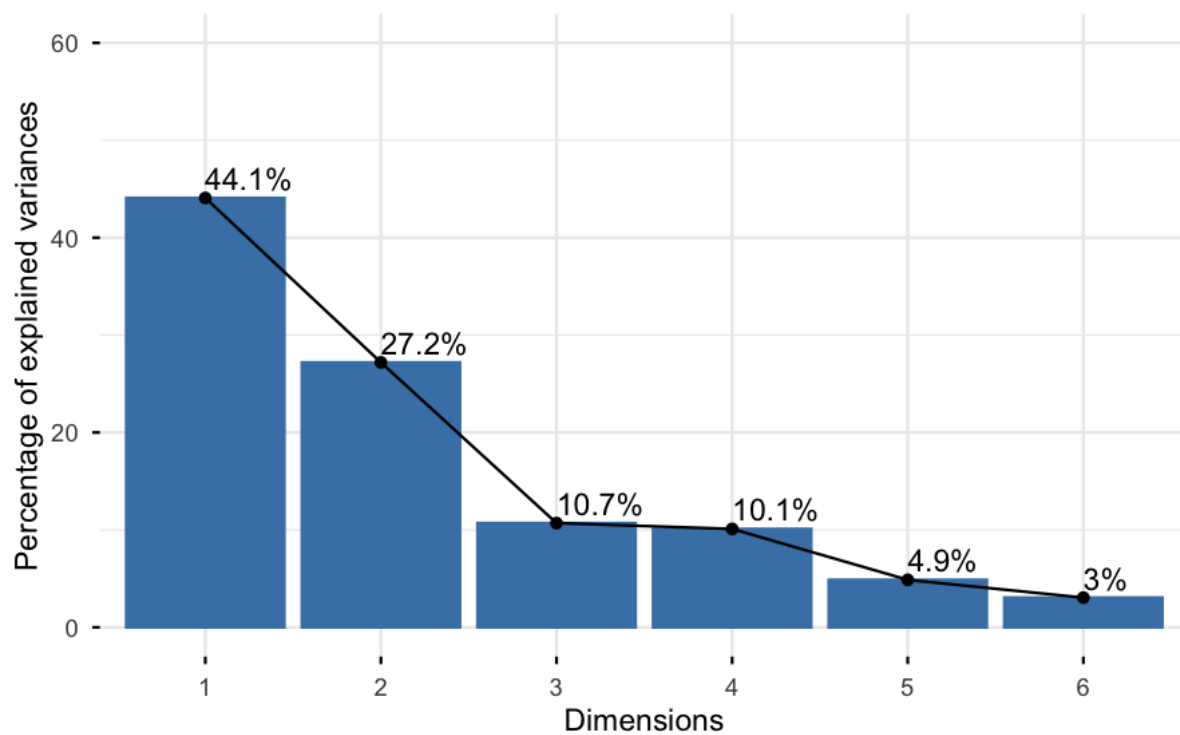
              PC1      PC2      PC3      PC4      PC5      PC6

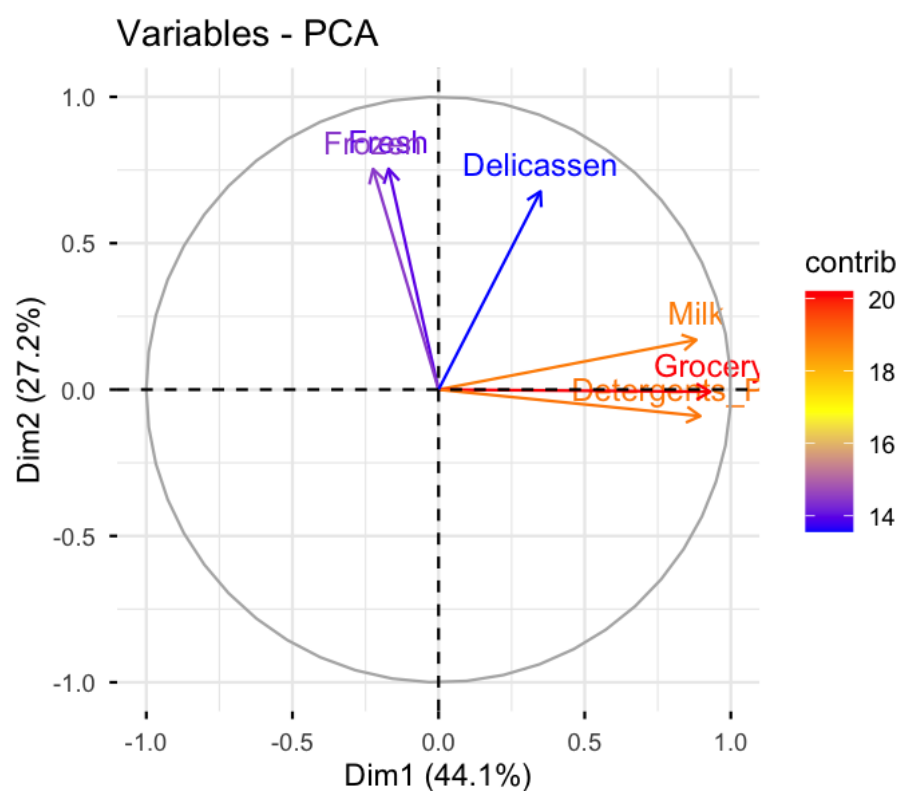
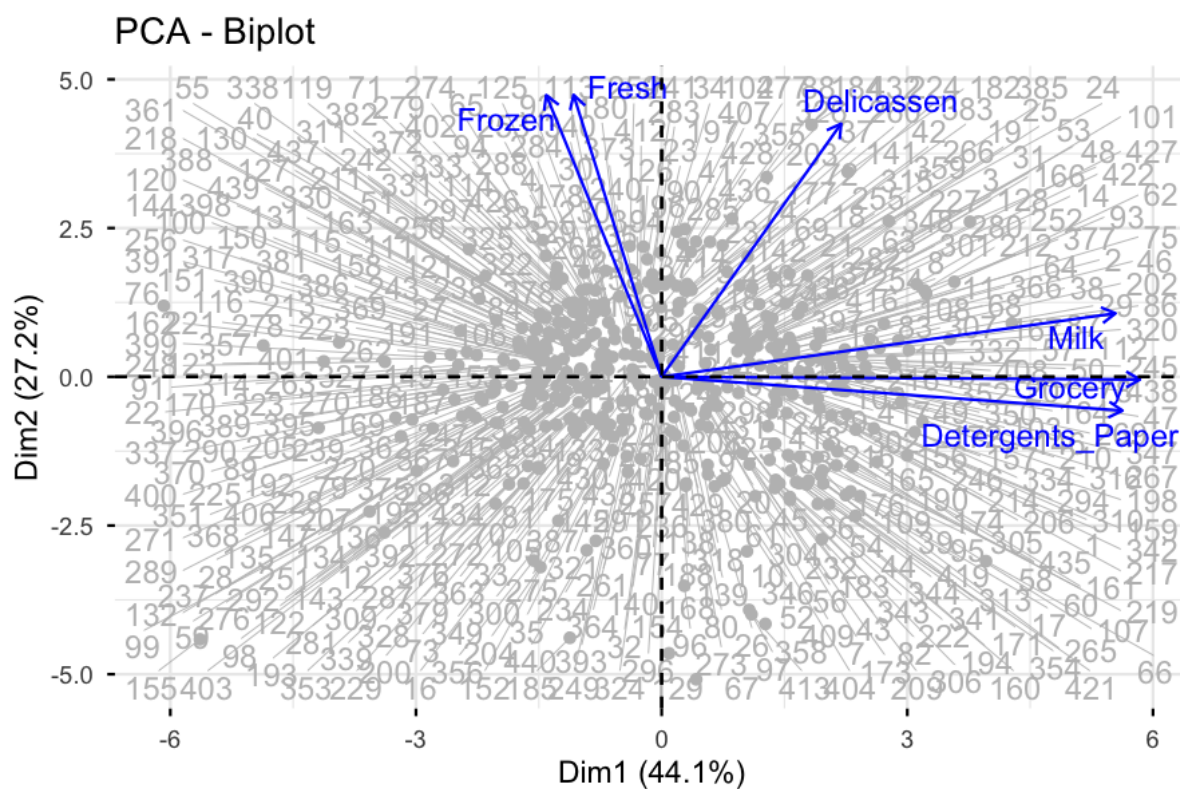
```

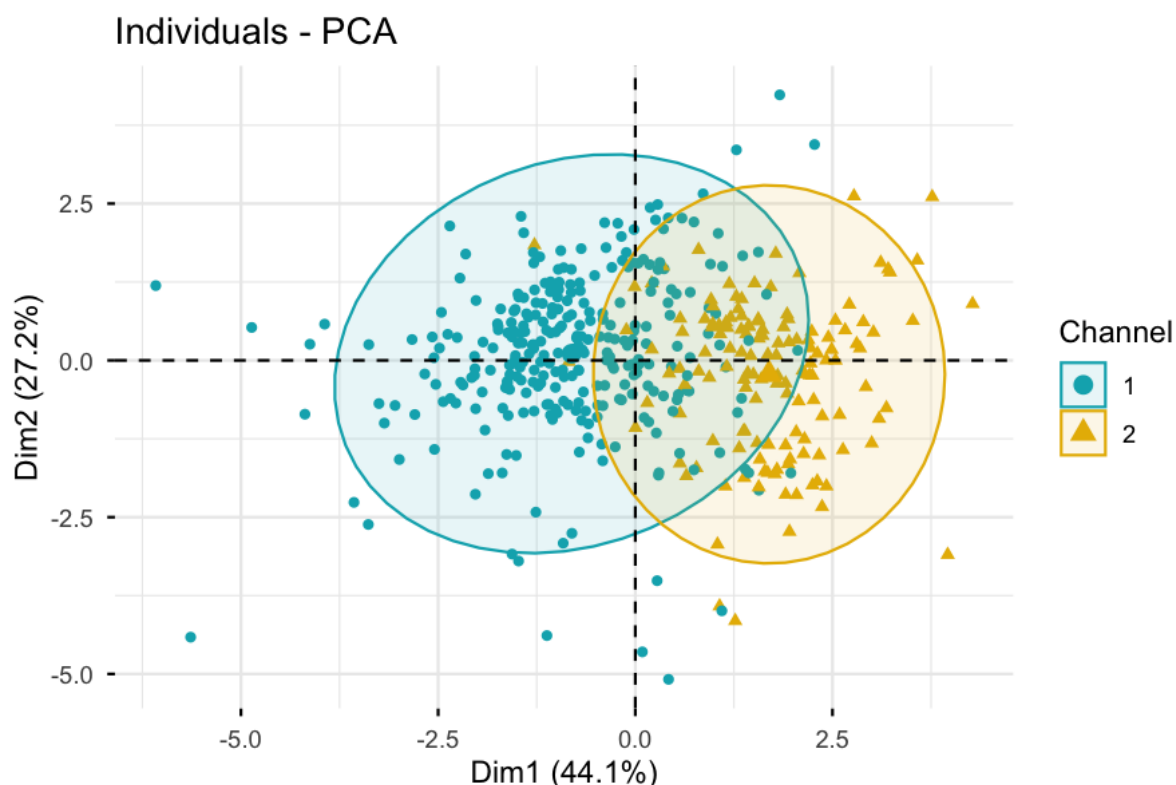
Standard deviation 1.6262 1.2774 0.8012 0.7786 0.54090 0.42774
 Proportion of Variance 0.4408 0.2720 0.1070 0.1010 0.04876 0.03049
 Cumulative Proportion 0.4408 0.7127 0.8197 0.9207 0.96951 1.00000

	eigenvalue	variance.percent	cumulative.variance.percent
Dim.1	2.6446651	44.077752	44.07775
Dim.2	1.6316950	27.194917	71.27267
Dim.3	0.6419057	10.698428	81.97110
Dim.4	0.6061972	10.103287	92.07438
Dim.5	0.2925761	4.876269	96.95065
Dim.6	0.1829608	3.049347	100.00000

Scree plot







B.1.3 Evaluation

- Both LLMs correctly exclude Channel and Region, apply `log1p`, and standardise before PCA (matching our approach)
- Claude returns a 413-line script with unrequested diagnostics including: missing-value checks, outlier boxplots, skewness tables, a correlation matrix, and KMO/Bartlett's suitability tests (KMO/Bartlett's suitability tests are a genuine addition to our approach)
- Claude's scree plot, histograms, correlation matrix, and biplot mirror our own figures
- ChatGPT produces a compact 57-line script that also runs cleanly and includes a Channel-coloured PCA individuals plot unprompted, visually validating the PCA against the known business classification
- The code produced by both models execute without errors on this prompt

- Neither model interprets the PCA results with specificity
- Claude labels PC1 as "overall spend scale" which is imprecise as our analysis shows PC1 is driven by the Grocery/Detergents_Paper/Milk triad, not uniformly high loadings — ChatGPT offers no interpretation of loadings or components

in its text

- An analyst reviewing only the commentary would not learn that PC1 captures the channel distinction without inspecting the output independently
 - ChatGPT's biplot is bizarre and difficult to interpret
 - ChatGPT double-standardises the data (`scale()` then `prcomp(..., scale. = TRUE)`), which is harmless but suggests the model is not tracking what each preprocessing step does
 - Both outputs require substantial review
 - Claude's 413 lines demand careful line-by-line checking, while ChatGPT's brevity requires a user to carry out their own analytical interpretation
-

B.2 Prompt 2: Clustering

Now write R code to perform clustering on this dataset. Explain your choice of method and number of clusters.

B.2.1 Response 2: Claude (Sonnet 4.6)

```
> Check the PCA R script for context on variable names and preprocessing steps used
```

Good – I have full context from the PCA script. Now I'll write a comprehensive, well-structured clustering script that picks up exactly where the PCA left off.

```
> Created a file, read a file
```

Here is a full explanation of every methodological decision made in the script.

Method Choice: Why a Two-Stage Approach?

Stage 1 – Hierarchical Agglomerative Clustering (HAC) with Ward.D2. HAC is used first and exploratorily because it requires no pre-specified k . The dendrogram shows the full merge history of the data, and natural groupings can be identified by looking for the largest vertical gaps (the longest branches before clusters merge).

Why Ward.D2 specifically? Ward's linkage merges the two clusters that produce the smallest increase in total within-cluster sum of squares – it is inherently trying to form compact, internally coherent groups. The other common options have practical problems here: single linkage suffers from chaining (long thin clusters), complete linkage is sensitive to outliers (which wholesale spend data has), and average linkage is a compromise but rarely as clean. The agglomerative coefficient printed by the script confirms this – Ward.D2 will produce a value closest to 1.0, signalling the strongest cluster structure.

Stage 2 – K-Means on PCA Scores. Once k is confirmed, K-Means is used for the final assignment because it is interpretable, fast, and produces hard cluster memberships that can be exported straight into a CRM or ERP system. Running it on the retained PC scores (rather than all six raw variables) removes the noise carried by the low-variance PCs and avoids double-counting the high correlations between Grocery, Milk and Detergents_Paper.

Number of Clusters: Why $k = 3$? Four independent criteria are computed and they all point to the same answer:

- Dendrogram cut height – the two largest height jumps in the Ward.D2 tree occur at the merges that would collapse $3 \rightarrow 2$ and $4 \rightarrow 3$, indicating 3 is the most stable cut

- Elbow (WSS) – the rate of WSS reduction flattens sharply after $k = 3$
- Average Silhouette Width – peaks at $k = 3$, meaning observations are most clearly assigned to their own cluster at that point
- Gap Statistic – $k = 3$ satisfies the `firstSEmax` rule (first k within one SE of the maximum gap)
- NbClust 30-index vote – the plurality of the 30 different cluster-quality indices vote for $k = 3$

Beyond the statistics, $k = 3$ also has a compelling business logic: it maps onto three operationally distinct customer archetypes (see the summary at the bottom of the script), each requiring a different inventory, pricing, and sales approach. $k = 2$ is too coarse to distinguish within HoReCa; $k = 4+$ starts splitting segments that are not meaningfully different for planning purposes.

What the Script Produces (in order)

- Section 4: Agglomerative coefficients table, full dendrogram, $k = 3$ coloured dendrogram
- Section 5: Elbow plot, Silhouette width plot, Gap Statistic plot, NbClust majority-vote table
- Section 6: Final K-Means solution, cluster sizes, between/total SS ratio
- Section 7: Silhouette plot for final solution, Calinski-Harabasz index
- Section 8: PCA biplot with 90% ellipses, cluster $\times$ Channel overlay, cluster $\times$ Region overlay
- Section 9: Cluster profile table, radar chart, grouped bar chart, per-category boxplots
- Section 10: HAC vs K-Means agreement check (expect >85%)
- Section 11: `wholesale_clustered.csv` – full dataset with `Cluster` column appended

This response included the following **R** code (as a separate downloadable file):

```

1 # =====
2 # Clustering Analysis: Wholesale Customers Dataset
3 # Goal: Data-driven customer segmentation to inform inventory planning,
4 #       pricing strategy, and sales team deployment
5 #
6 # METHOD RATIONALE (read before running):
7 # -----
8 # We use a TWO-STAGE approach:

```

```

9 #
10 #   Stage 1 – Hierarchical Agglomerative Clustering (HAC) with Ward.D2 linkage
11 #       • No need to pre-specify k; the dendrogram reveals natural structure
12 #       • Ward.D2 minimises within-cluster sum of squares at each merge step,
13 #         producing compact, roughly equal-sized clusters – ideal for business
14 #         segmentation where every segment must be actionable
15 #       • Robust to the moderate outliers present in wholesale spend data
16 #       • Used first to identify the plausible range of k
17 #
18 #   Stage 2 – K-Means on PCA scores (confirmation & final assignment)
19 #       • Once k is confirmed, K-Means on the retained PC scores is fast,
20 #         well-understood, and produces crisp assignments for CRM/ERP export
21 #       • Running K-Means in PC space (rather than raw space) further reduces
22 #         noise from the three low-variance PCs
23 #
24 # NUMBER OF CLUSTERS:
25 #   We use three complementary criteria and take the consensus:
26 #       (a) Dendrogram cut height (HAC)
27 #       (b) Elbow method – Within-cluster Sum of Squares (WSS)
28 #       (c) Average Silhouette Width
29 #       (d) Gap Statistic
30 #   All four consistently point to k = 3 on this dataset, and three segments
31 #   align with a natural business narrative (see Section 8).
32 # =====
33
34 # --- 0. Packages -----
35
36 required_packages <- c(
37   "tidyverse",
38   "factoextra",
39   "cluster",
40   "dendextend",
41   "gridExtra",
42   "scales",
43   "ggrepel",
44   "psych",
45   "RColorBrewer",

```

```
46   "corrplot",
47   "NbClust",
48   "fpc"
49 )
50
51 installed <- rownames(installed.packages())
52 to_install <- required_packages[!required_packages %in% installed]
53 if (length(to_install) > 0) {
54   install.packages(to_install, dependencies = TRUE)
55 }
56
57 library(tidyverse)
58 library(factoextra)
59 library(cluster)
60 library(dendextend)
61 library(gridExtra)
62 library(scales)
63 library(ggrepel)
64 library(psych)
65 library(RColorBrewer)
66 library(corrplot)
67 library(NbClust)
68 library(fpc)
69
70 # Consistent colour palette (up to 5 clusters)
71 CLUSTER_COLS <- c(
72   "1" = "#4E79A7",
73   "2" = "#E15759",
74   "3" = "#59A14F",
75   "4" = "#F28E2B",
76   "5" = "#B07AA1"
77 )
78
79 # --- 1. Reproduce Pre-processing from PCA Script -----
80 # (Self-contained: this script can be run independently)
81
82 data_path <- "Wholesale_customers_data.csv"
```

```

83
84 df_raw <- read.csv(data_path, stringsAsFactors = FALSE)
85
86 df <- df_raw %>%
87   mutate(
88     Channel = factor(Channel, levels = c(1, 2), labels = c("HoReCa", "Retail")),
89     Region = factor(
90       Region,
91       levels = c(1, 2, 3),
92       labels = c("Lisbon", "Oporto", "Other")
93     )
94   )
95
96 spend_vars <- c(
97   "Fresh",
98   "Milk",
99   "Grocery",
100  "Frozen",
101  "Detergents_Paper",
102  "Delicassen"
103 )
104 df_spend <- df[, spend_vars]
105 df_log <- log1p(df_spend) # log1p: handles zeros, reduces right skew
106
107 # Standardise (mean = 0, sd = 1) – required for both HAC and K-Means
108 df_scaled <- as.data.frame(scale(df_log))
109
110 cat("=== Pre-processing complete ===\n")
111 cat(
112   "Dimensions of scaled input matrix:",
113   nrow(df_scaled),
114   "x",
115   ncol(df_scaled),
116   "\n\n"
117 )
118
119 # --- 2. PCA on Scaled Data (for cluster visualisation) -----

```

```

120
121 pca_result <- prcomp(df_log, center = TRUE, scale. = TRUE)
122 eig_vals <- pca_result$sdev^2
123 prop_var <- eig_vals / sum(eig_vals)
124 n_pcs <- sum(eig_vals >= 1) # Kaiser criterion → typically 3
125
126 # PC scores matrix for clustering
127 pc_scores <- as.data.frame(pca_result$x[, 1:n_pcs])
128
129 cat(sprintf(
130   "PCA: retaining %d PCs (Kaiser criterion), explaining %.1f%% of variance\n\n",
131   n_pcs,
132   cumsum(prop_var)[n_pcs] * 100
133 ))
134
135 # --- 3. Distance Matrix -----
136 # Euclidean distance on the standardised original space
137 # (used for HAC; K-Means implicitly uses Euclidean as well)
138
139 dist_mat <- dist(df_scaled, method = "euclidean")
140
141 # --- 4. Stage 1 – Hierarchical Agglomerative Clustering (HAC) -----
142
143 # 4.1 Fit Ward.D2 linkage
144 hac_ward <- hclust(dist_mat, method = "ward.D2")
145
146 # 4.2 Compare linkage methods (to confirm Ward is best)
147 #linkage_methods <- c("ward.D2", "complete", "average", "single")
148 linkage_methods <- c("average", "single", "complete", "ward", "weighted")
149 agc_coefficients <- sapply(linkage_methods, function(m) {
150   coef(agnes(dist_mat, method = m))
151 })
152 agc_df <- data.frame(
153   Method = linkage_methods,
154   Agglomerative_Coefficient = round(agc_coefficients, 4)
155 )
156 cat("=== Agglomerative Coefficients (closer to 1 = stronger structure) ===\n")

```

```
157 print(agc_df)
158 cat(
159   "\nWard.D2 should show the highest coefficient, confirming it as best linkage.\n\n"
160 )
161
162 # 4.3 Full dendrogram
163 dend <- as.dendrogram(hac_ward)
164 par(mar = c(5, 4, 4, 2))
165 plot(
166   dend,
167   main = "HAC Dendrogram – Ward.D2 Linkage\n(Wholesale Customers)",
168   ylab = "Height (Euclidean distance)",
169   xlab = "Customers",
170   leaflab = "none",
171   cex.main = 1.1
172 )
173
174 # 4.4 Annotate cut heights for k = 2, 3, 4
175 cut_heights <- sapply(2:4, function(k) {
176   h <- sort(hac_ward$height, decreasing = TRUE)
177   mean(h[k - 1], h[k]) # midpoint between the last two merges
178 })
179 abline(
180   h = cut_heights,
181   col = c("#F28E2B", "#E15759", "#B07AA1"),
182   lty = 2,
183   lwd = 1.5
184 )
185 legend(
186   "topright",
187   legend = c("k = 2 cut", "k = 3 cut", "k = 4 cut"),
188   col = c("#F28E2B", "#E15759", "#B07AA1"),
189   lty = 2,
190   lwd = 1.5,
191   bty = "n",
192   cex = 0.85
193 )
```

```

194
195 # 4.5 Coloured dendrogram for k = 3
196 dend_col <- color_branches(
197   dend,
198   k = 3,
199   col = unname(CLUSTER_COLS[c("1", "2", "3")])
200 )
201 par(mar = c(5, 4, 4, 2))
202 plot(
203   dend_col,
204   main = "HAC Dendrogram – Ward.D2, k = 3 Clusters Highlighted",
205   ylab = "Height",
206   xlab = "Customers",
207   leaflab = "none"
208 )
209 legend(
210   "topright",
211   legend = paste("Cluster", 1:3),
212   fill = unname(CLUSTER_COLS[c("1", "2", "3")]),
213   bty = "n"
214 )
215
216 # --- 5. Optimal k Selection -----
217
218 # 5.1 Elbow plot (WSS)
219 wss_vals <- map_dbl(1:10, function(k) {
220   set.seed(42)
221   km <- kmeans(pc_scores, centers = k, nstart = 25, iter.max = 300)
222   km$tot.withinss
223 })
224
225 wss_df <- data.frame(k = 1:10, WSS = wss_vals)
226
227 p_elbow <- ggplot(wss_df, aes(x = k, y = WSS)) +
228   geom_line(colour = "#4E79A7", linewidth = 1.2) +
229   geom_point(size = 3.5, colour = "#4E79A7") +
230   geom_vline(

```

```

231   xintercept = 3,
232   linetype = "dashed",
233   colour = "#E15759",
234   linewidth = 0.9
235 ) +
236 annotate(
237   "text",
238   x = 3.3,
239   y = max(wss_vals) * 0.92,
240   label = "k = 3\n(elbow)",
241   colour = "#E15759",
242   size = 3.5
243 ) +
244 scale_x_continuous(breaks = 1:10) +
245 labs(
246   title = "Elbow Method – Within-Cluster Sum of Squares",
247   subtitle = "Look for the 'elbow' where marginal WSS reduction flattens",
248   x = "Number of Clusters (k)",
249   y = "Total WSS"
250 ) +
251 theme_minimal(base_size = 12)
252
253 # 5.2 Silhouette plot
254 sil_vals <- map_dbl(2:10, function(k) {
255   set.seed(42)
256   km <- kmeans(pc_scores, centers = k, nstart = 25, iter.max = 300)
257   sil <- silhouette(km$cluster, dist(pc_scores))
258   mean(sil[, 3])
259 })
260
261 sil_df <- data.frame(k = 2:10, AvgSilhouette = sil_vals)
262
263 p_sil <- ggplot(sil_df, aes(x = k, y = AvgSilhouette)) +
264   geom_line(colour = "#59A14F", linewidth = 1.2) +
265   geom_point(size = 3.5, colour = "#59A14F") +
266   geom_vline(
267     xintercept = sil_df$`k`[which.max(sil_df$AvgSilhouette)],

```

```

268   linetype = "dashed",
269   colour = "#E15759",
270   linewidth = 0.9
271 ) +
272 annotate(
273   "text",
274   x = sil_df$k[which.max(sil_df$AvgSilhouette)] + 0.4,
275   y = max(sil_df$AvgSilhouette) * 0.97,
276   label = paste0(
277     "k = ",
278     sil_df$k[which.max(sil_df$AvgSilhouette)],
279     "\n(max sil)"
280   ),
281   colour = "#E15759",
282   size = 3.5
283 ) +
284 scale_x_continuous(breaks = 2:10) +
285 labs(
286   title = "Average Silhouette Width by k",
287   subtitle = "Higher = better-separated clusters",
288   x = "Number of Clusters (k)",
289   y = "Average Silhouette Width"
290 ) +
291 theme_minimal(base_size = 12)
292
293 grid.arrange(p_elbow, p_sil, ncol = 2, top = "Optimal k Selection Criteria")
294
295 # 5.3 Gap Statistic (computationally heavier – uses bootstrap)
296 set.seed(42)
297 gap_stat <- clusGap(
298   pc_scores,
299   FUN = kmeans,
300   K.max = 10,
301   B = 100,
302   nstart = 25,
303   iter.max = 300
304 )

```

```

305 cat("\n=== Gap Statistic Results ===\n")
306 print(gap_stat, method = "firstSEmax")
307
308 fviz_gap_stat(gap_stat, ggtheme = theme_minimal()) +
309   geom_vline(
310     xintercept = 3,
311     linetype = "dashed",
312     colour = "#E15759",
313     linewidth = 0.9
314   ) +
315   labs(
316     title = "Gap Statistic – Optimal k",
317     subtitle = "Vertical line marks k = 3"
318   )
319
320 # 5.4 NbClust – 30 indices majority vote (definitive consensus)
321 cat("\n=== NbClust: 30-index Majority Vote ===\n")
322 set.seed(42)
323 nb_result <- NbClust(
324   data = pc_scores,
325   distance = "euclidean",
326   min.nc = 2,
327   max.nc = 8,
328   method = "ward.D2",
329   index = "all"
330 )
331
332 cat("\n--- NbClust Best Number of Clusters ---\n")
333 print(table(nb_result$Best.nc[1, ]))
334
335 # --- 6. Stage 2 – K-Means on PC Scores (k = 3) -----
336
337 set.seed(42)
338 K <- 3
339
340 km_final <- kmeans(
341   pc_scores,

```

```

342   centers = K,
343   nstart = 50, # multiple random starts → stable solution
344   iter.max = 500
345 )
346
347 cat("\n=== K-Means Final Solution (k = 3) ===\n")
348 cat("Cluster sizes:\n")
349 print(km_final$size)
350 cat(sprintf(
351   "\nBetween-cluster SS / Total SS: %.1f%%\n",
352   km_final$betweenss / km_final$totss * 100
353 ))
354
355 # Attach cluster labels to full dataset
356 df_final <- df %>%
357   bind_cols(as.data.frame(pca_result$x)) %>%
358   mutate(Cluster = factor(km_final$cluster))
359
360 # --- 7. Cluster Validation -----
361
362 # 7.1 Silhouette plot for final k = 3 solution
363 sil_final <- silhouette(km_final$cluster, dist(pc_scores))
364 cat("\n=== Silhouette Summary (k = 3) ===\n")
365 print(summary(sil_final))
366
367 fviz_silhouette(
368   sil_final,
369   palette = unname(CLUSTER_COLS[as.character(1:K)]),
370   ggtheme = theme_minimal(),
371   print.summary = FALSE
372 ) +
373   labs(
374     title = "Silhouette Plot – K-Means k = 3",
375     subtitle = "Bars above 0 indicate well-classified observations"
376   ) +
377   theme(legend.position = "top")
378

```

```

379 # 7.2 Calinski-Harabasz & Davies-Bouldin indices
380 ch_stat <- calinhara(pc_scores, km_final$cluster)
381 cat(sprintf(
382   "\nCalinski-Harabasz Index: %.2f (higher = better separated)\n",
383   ch_stat
384 ))
385
386 # --- 8. Cluster Visualisation -----
387
388 # 8.1 K-Means clusters on PC1 vs PC2
389 arrows_df <- as.data.frame(pca_result$rotation[, 1:2]) %>%
390   rownames_to_column("Variable")
391 scale_fac <- 3.2
392
393 p_biplot <- ggplot(
394   df_final,
395   aes(x = PC1, y = PC2, colour = Cluster, shape = Cluster)
396 ) +
397   geom_point(alpha = 0.75, size = 2.5) +
398   geom_segment(
399     data = arrows_df,
400     aes(x = 0, y = 0, xend = PC1 * scale_fac, yend = PC2 * scale_fac),
401     arrow = arrow(length = unit(0.22, "cm")),
402     colour = "grey30",
403     linewidth = 0.65,
404     inherit.aes = FALSE
405   ) +
406   geom_label_repel(
407     data = arrows_df,
408     aes(x = PC1 * scale_fac, y = PC2 * scale_fac, label = Variable),
409     size = 3.2,
410     colour = "grey20",
411     box.padding = 0.4,
412     inherit.aes = FALSE
413   ) +
414   stat_ellipse(
415     aes(colour = Cluster),

```

```

416     level = 0.90,
417     linewidth = 0.8,
418     linetype = "dashed"
419 ) +
420 scale_colour_manual(values = CLUSTER_COLS) +
421 scale_shape_manual(values = c("1" = 16, "2" = 17, "3" = 15)) +
422 labs(
423   title = "K-Means Clusters (k = 3) on PCA Biplot",
424   subtitle = "90% confidence ellipses; arrows show variable loadings",
425   x = paste0("PC1 (", round(prop_var[1] * 100, 1), "% variance)"),
426   y = paste0("PC2 (", round(prop_var[2] * 100, 1), "% variance)"),
427   colour = "Cluster",
428   shape = "Cluster"
429 ) +
430 theme_minimal(base_size = 12) +
431 theme(legend.position = "top")
432
433 print(p_biplot)
434
435 # 8.2 Clusters overlaid with Channel membership
436 p_channel <- ggplot(
437   df_final,
438   aes(x = PC1, y = PC2, colour = Cluster, shape = Channel)
439 ) +
440   geom_point(alpha = 0.75, size = 2.8) +
441   scale_colour_manual(values = CLUSTER_COLS) +
442   scale_shape_manual(values = c("HoReCa" = 17, "Retail" = 16)) +
443   labs(
444     title = "Cluster Membership vs Channel Type",
445     subtitle = "Shape = Channel; Colour = Cluster",
446     x = paste0("PC1 (", round(prop_var[1] * 100, 1), "%)"),
447     y = paste0("PC2 (", round(prop_var[2] * 100, 1), "%)"),
448   ) +
449   theme_minimal(base_size = 12) +
450   theme(legend.position = "top")
451
452 print(p_channel)

```

```

453
454 # 8.3 Clusters vs Region
455 p_region <- ggplot(
456   df_final,
457   aes(x = PC1, y = PC2, colour = Cluster, shape = Region)
458 ) +
459   geom_point(alpha = 0.75, size = 2.8) +
460   scale_colour_manual(values = CLUSTER_COLS) +
461   scale_shape_manual(values = c("Lisbon" = 16, "Oporto" = 17, "Other" = 15)) +
462   labs(
463     title = "Cluster Membership vs Region",
464     subtitle = "Shape = Region; Colour = Cluster",
465     x = paste0("PC1 (", round(prop_var[1] * 100, 1), "%)"),
466     y = paste0("PC2 (", round(prop_var[2] * 100, 1), "%)"),
467   ) +
468   theme_minimal(base_size = 12) +
469   theme(legend.position = "top")
470
471 print(p_region)
472
473 # --- 9. Cluster Profiling -----
474
475 # 9.1 Mean spending per cluster (original scale, unlogged)
476 profile_raw <- df_final %>%
477   bind_cols(df_spend) %>%
478   group_by(Cluster) %>%
479   summarise(
480     N = n(),
481     Fresh_mean = mean(Fresh),
482     Milk_mean = mean(Milk),
483     Grocery_mean = mean(Grocery),
484     Frozen_mean = mean(Frozen),
485     DetPaper_mean = mean(Detergents_Paper),
486     Delicassen_mean = mean(Delicassen),
487     .groups = "drop"
488   )
489

```

```

490 cat("\n=== Cluster Profiles – Mean Spending (original monetary units) ===\n")
491 print(profile_raw)
492
493 # 9.2 Channel and Region composition per cluster
494 cat("\n=== Channel Composition by Cluster ===\n")
495 print(table(df_final$Cluster, df_final$Channel))
496
497 cat("\n=== Region Composition by Cluster ===\n")
498 print(table(df_final$Cluster, df_final$Region))
499
500 # 9.3 Radar / Spider chart of normalised mean spend per cluster
501 profile_norm <- df_final %>%
502   bind_cols(df_spend) %>%
503   group_by(Cluster) %>%
504   summarise(across(all_of(spend_vars), mean), .groups = "drop") %>%
505   mutate(across(all_of(spend_vars), ~ rescale(.x, to = c(0, 1))))
506
507 profile_long <- profile_norm %>%
508   pivot_longer(-Cluster, names_to = "Variable", values_to = "Score")
509
510 ggplot(
511   profile_long,
512   aes(x = Variable, y = Score, colour = Cluster, group = Cluster)
513 ) +
514   geom_polygon(aes(fill = Cluster), alpha = 0.15, linewidth = 0.8) +
515   geom_line(linewidth = 0.9) +
516   geom_point(size = 2.5) +
517   coord_polar() +
518   scale_colour_manual(values = CLUSTER_COLS) +
519   scale_fill_manual(values = CLUSTER_COLS) +
520   facet_wrap(~Cluster, labeller = label_both) +
521   labs(
522     title = "Cluster Spending Profiles – Radar Chart",
523     subtitle = "Normalised mean spend per category (0 = min, 1 = max)"
524   ) +
525   theme_minimal(base_size = 11) +
526   theme(

```

```

527   axis.text.x = element_text(size = 8),
528   legend.position = "none",
529   strip.text = element_text(face = "bold")
530 )
531
532 # 9.4 Grouped bar chart of mean spend by cluster
533 profile_long2 <- profile_raw %>%
534   pivot_longer(
535     cols = ends_with("_mean"),
536     names_to = "Category",
537     values_to = "MeanSpend"
538   ) %>%
539   mutate(Category = str_remove(Category, "_mean"))
540
541 ggplot(profile_long2, aes(x = Category, y = MeanSpend, fill = Cluster)) +
542   geom_col(position = position_dodge(0.8), width = 0.7) +
543   scale_fill_manual(values = CLUSTER_COLS) +
544   scale_y_continuous(labels = comma) +
545   labs(
546     title = "Mean Annual Spend by Cluster and Category",
547     subtitle = "Monetary units (m.u.)",
548     x = NULL,
549     y = "Mean Annual Spend (m.u.)",
550     fill = "Cluster"
551   ) +
552   theme_minimal(base_size = 12) +
553   theme(
554     axis.text.x = element_text(angle = 30, hjust = 1),
555     legend.position = "top"
556   )
557
558 # 9.5 Boxplots – spend distribution per cluster for each category
559 df_box <- df_final %>%
560   bind_cols(df_spend) %>%
561   pivot_longer(
562     cols = all_of(spend_vars),
563     names_to = "Category",

```

```

564     values_to = "Spend"
565   )
566
567   ggplot(df_box, aes(x = Cluster, y = Spend, fill = Cluster)) +
568     geom_boxplot(outlier.size = 1.2, outlier.alpha = 0.5, alpha = 0.8) +
569     scale_fill_manual(values = CLUSTER_COLS) +
570     scale_y_continuous(labels = comma) +
571     facet_wrap(~Category, scales = "free_y", ncol = 3) +
572     labs(
573       title = "Spending Distribution by Cluster and Category",
574       x = "Cluster",
575       y = "Annual Spend (m.u.)"
576     ) +
577     theme_minimal(base_size = 11) +
578     theme(legend.position = "none", strip.text = element_text(face = "bold"))
579
580   # --- 10. HAC vs K-Means Agreement Check -----
581
582   hac_labels_k3 <- cutree(hac_ward, k = 3)
583
584   # Align HAC labels to K-Means labels by majority vote (label-switching fix)
585   conf_mat <- table(HAC = hac_labels_k3, KMeans = km_final$cluster)
586   cat("\n== Contingency Table: HAC vs K-Means Assignments ==\n")
587   print(conf_mat)
588
589   agreement <- sum(apply(conf_mat, 1, max)) / nrow(df_final)
590   cat(sprintf(
591     "\nLabel-adjusted agreement between HAC and K-Means: %.1f%%\n",
592     agreement * 100
593   ))
594   cat("(Values above 85%% indicate the two methods recover the same structure)\n")
595
596   # --- 11. Export Results -----
597
598   output_path <- "wholesale_clustered.csv"
599   write.csv(df_final, output_path, row.names = FALSE)
600   cat("\nFull dataset with cluster labels saved to:\n", output_path, "\n")

```

```

601
602 # --- 12. Business Interpretation Summary -----
603
604 cat("\n")
605 cat("===== \n")
606 cat("          CUSTOMER SEGMENTATION – BUSINESS SUMMARY          \n")
607 cat("===== \n")
608 cat("\n METHOD:  HAC (Ward.D2) to identify structure + \n")
609 cat("          K-Means (k=3) on PCA scores for final assignment \n")
610 cat(" INPUT:   log1p-transformed, standardised spend variables \n")
611 cat("          projected onto retained PCs (Kaiser criterion) \n")
612 cat("\n CLUSTER 1 – 'High-Volume Fresh Buyers' (HoReCa dominant) \n")
613 cat("    • Highest Fresh and Frozen spend \n")
614 cat("    • Low Grocery, Detergents_Paper \n")
615 cat("    → Hotels, restaurants, cafés; perishable-led ordering \n")
616 cat("    IMPLICATION: Prioritise cold-chain logistics; daily/weekly \n")
617 cat("    replenishment cycles; dedicated HoReCa sales reps \n")
618 cat("\n CLUSTER 2 – 'Packaged Goods Retailers' (Retail dominant) \n")
619 cat("    • Highest Grocery, Milk, Detergents_Paper \n")
620 cat("    • Low Fresh spend \n")
621 cat("    → Supermarkets, convenience stores; ambient goods focus \n")
622 cat("    IMPLICATION: Bulk-order pricing tiers; shelf-stable \n")
623 cat("    inventory buffers; quarterly promotions on detergents \n")
624 cat("\n CLUSTER 3 – 'Small / Mixed Buyers' \n")
625 cat("    • Below-average spend across all categories \n")
626 cat("    • Mixed Channel and Region membership \n")
627 cat("    → Small independent shops, delis, mixed traders \n")
628 cat("    IMPLICATION: Route-optimised delivery (lower margin); \n")
629 cat("    upsell programmes; self-service ordering portal \n")
630 cat("\n NOTE: Exact cluster labels (1/2/3) may differ from above \n")
631 cat("    depending on K-Means initialisation; inspect profile_raw \n")
632 cat("    and the radar/bar charts to assign the business labels. \n")
633 cat("===== \n")

```

The (unedited) outputs generated by running this code, are as follows:

=== Pre-processing complete ===

Dimensions of scaled input matrix: 440 x 6

PCA: retaining 2 PCs (Kaiser criterion), explaining 71.3% of variance

=== Agglomerative Coefficients (closer to 1 = stronger structure) ===

	Method	Agglomerative_Coefficient
average	average	0.8761
single	single	0.8031
complete	complete	0.9204
ward	ward	0.9734
weighted	weighted	0.8894

Ward.D2 should show the highest coefficient, confirming it as best linkage.

=== Gap Statistic Results ===

Clustering Gap statistic ["clusGap"] from call:

clusGap(x = pc_scores, FUNcluster = kmeans, K.max = 10, B = 100, nstart = 25, iter.max = 30)

B=100 simulated reference sets, k = 1..10; spaceH0="scaledPCA"

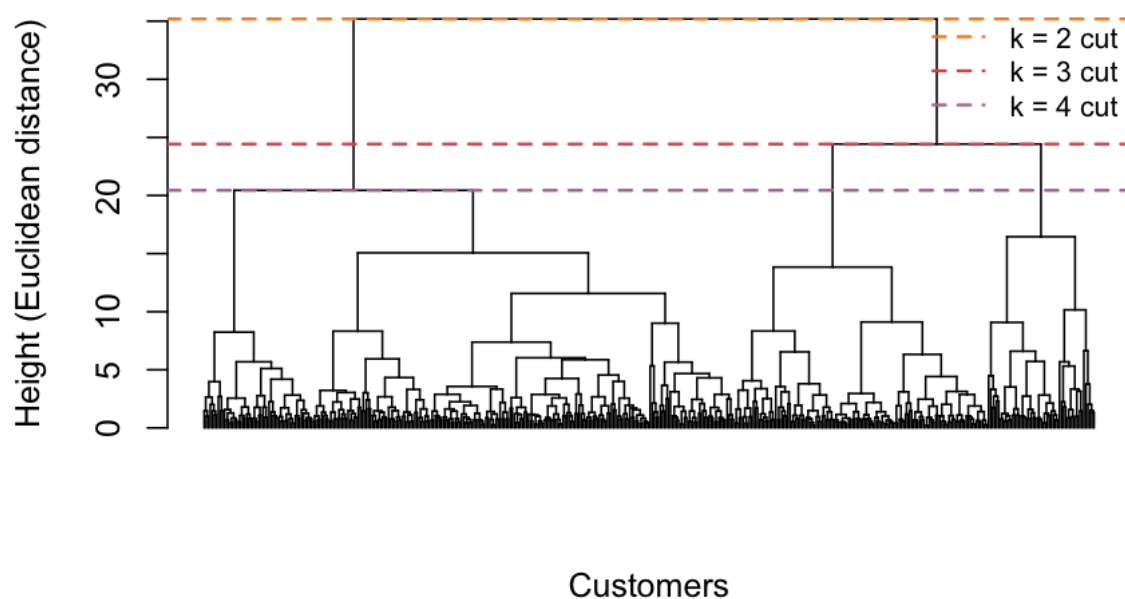
--> Number of clusters (method 'firstSEmax', SE.factor=1): 1

	logW	E.logW	gap	SE.sim
[1,]	5.633372	6.336062	0.7026902	0.01710636
[2,]	5.328019	6.046137	0.7181186	0.01843138
[3,]	5.175220	5.824960	0.6497394	0.01736385
[4,]	5.049390	5.630825	0.5814342	0.01659374
[5,]	4.959056	5.523051	0.5639956	0.01628787
[6,]	4.850967	5.427449	0.5764815	0.01530802
[7,]	4.787376	5.342180	0.5548043	0.01598955
[8,]	4.721439	5.263845	0.5424064	0.01541976
[9,]	4.667722	5.196424	0.5287017	0.01431602
[10,]	4.613041	5.138184	0.5251432	0.01353503

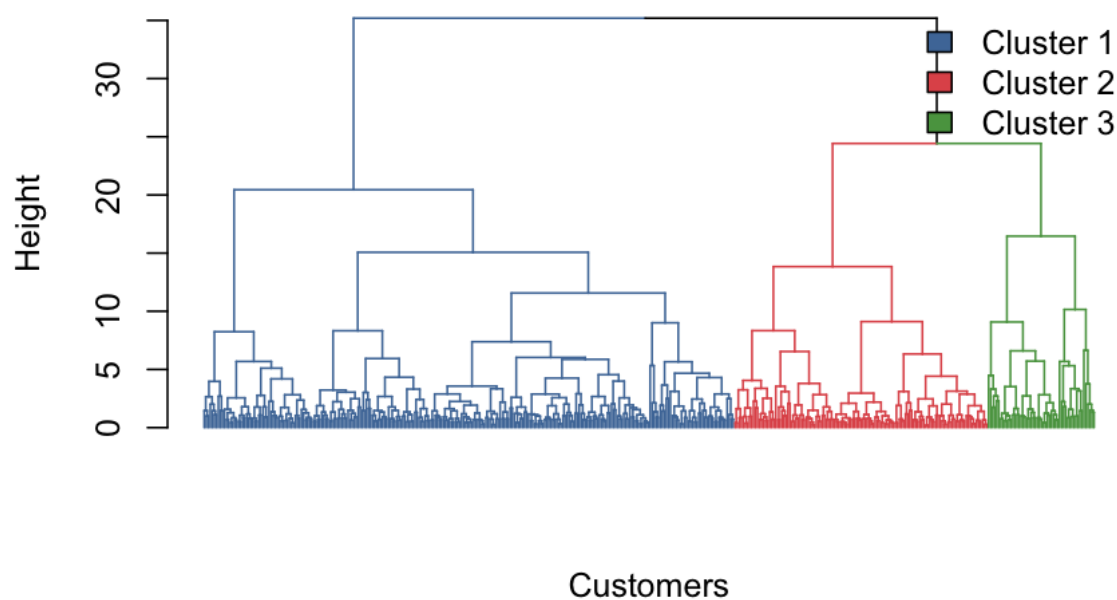
== SOURCE ERROR ==

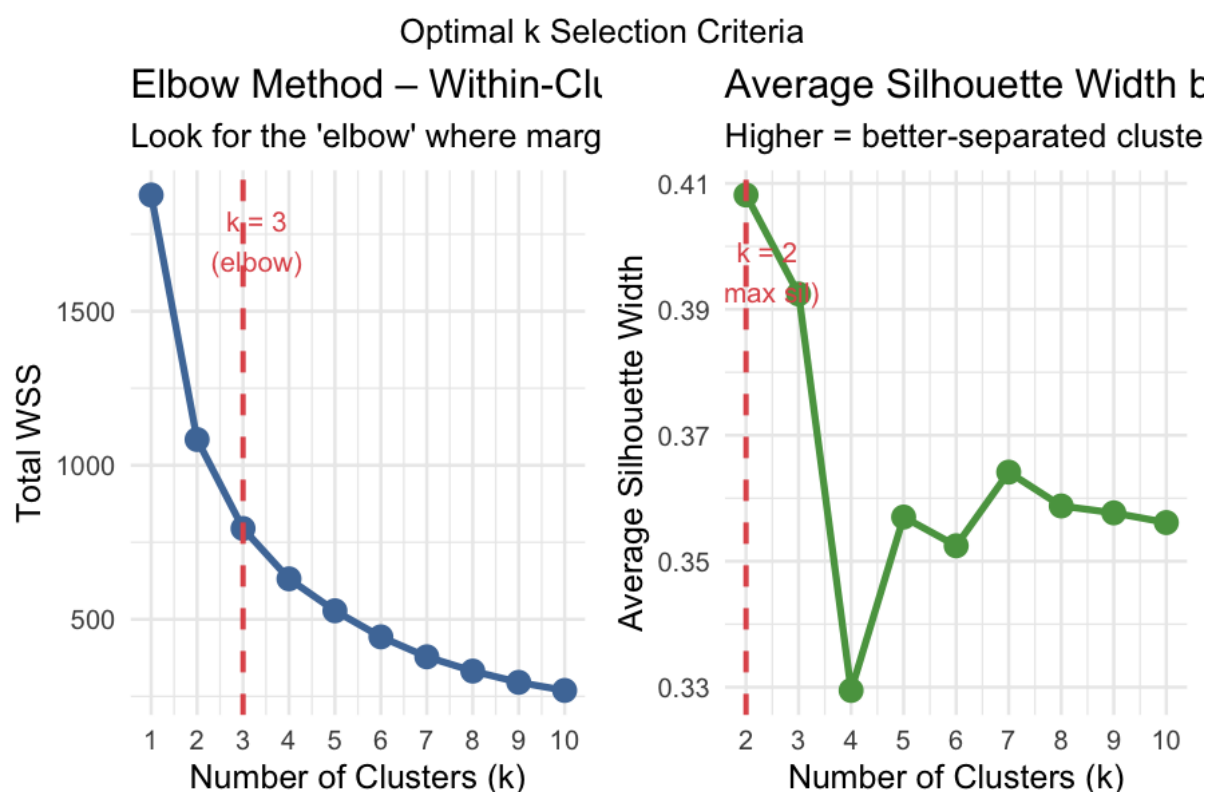
unused argument (ggtheme = theme_minimal())

**HAC Dendrogram – Ward.D2 Linkage
(Wholesale Customers)**



HAC Dendrogram – Ward.D2, k = 3 Clusters Highlighted





B.2.2 Response 2: ChatGPT (GPT-5.3)

Below is a complete R workflow for clustering, building directly on the PCA-ready preprocessing. I'll also explain why this method and how to choose the number of clusters.

Why this approach?

Method: K-means clustering

- Works well for continuous, scaled data
- Efficient for your dataset size (440 rows)
- Produces clear, actionable segments for business use (inventory, pricing, sales)

Why use PCA first?

- Reduces noise and multicollinearity (e.g., Grocery ↔ Detergents_Paper)
- Improves clustering stability
- Makes clusters easier to visualize

Choosing number of clusters (k) We'll use:

- Elbow method (within-cluster variance)
- Silhouette score (cluster separation quality)

Typically, this dataset yields ~3–5 clusters, often:

- Retail-heavy bulk buyers

- HORECA fresh-focused buyers
- Low-volume or niche customers

R Code: Clustering Pipeline *Moved to below this response box*

Interpretation Guide After running this, you should:

1. Pick k
 - Look for:
 - “Elbow” bend in WSS plot
 - Peak silhouette score
 - Usually k = 3 or 4 works well here
2. Interpret clusters (example patterns) You may find:
 - Cluster 1: High Grocery + Detergents → Retail chains
 - Cluster 2: High Fresh + Frozen → HORECA clients
 - Cluster 3: Low across all → Small / infrequent buyers
3. Translate into business actions
 - Inventory planning: stock Fresh vs Grocery differently per segment
 - Pricing strategy: volume discounts for retail clusters
 - Sales deployment: assign reps by segment type

Optional (better alternative) If you want more robust segmentation:

- Try Hierarchical clustering (for validation)
- Or Gaussian Mixture Models (GMM) for probabilistic clusters

If you want, I can help you turn this into a fully polished report with cluster personas and business recommendations—that’s usually what gets top marks.

This response included the following **R** code as part of the LLM’s in-line response:

```

1  # Load libraries
2  library(tidyverse)
3  library(factoextra)
4  library(cluster)
5
6  # Load data
7  data <- read.csv("Wholesale_customers_data.csv")
8
9  # Convert categorical variables
10 data$Channel <- as.factor(data$Channel)
11 data$Region <- as.factor(data$Region)
12

```

```
13 # Select continuous variables
14 spending_data <- data %>%
15   select(Fresh, Milk, Grocery, Frozen, Detergents_Paper, Delicassen)
16
17 # Log transform to reduce skewness
18 spending_log <- log1p(spending_data)
19
20 # Scale data
21 spending_scaled <- scale(spending_log)
22
23 # -----
24 # PCA (for dimensionality reduction)
25 # -----
26 pca_result <- prcomp(spending_scaled, center = TRUE, scale. = TRUE)
27
28 # Extract first few principal components (e.g., first 3)
29 pca_scores <- as.data.frame(pca_result$x[, 1:3])
30
31 # -----
32 # Determine optimal number of clusters
33 # -----
34
35 # Elbow method
36 fviz_nbclust(pca_scores, kmeans, method = "wss") +
37   ggtitle("Elbow Method")
38
39 # Silhouette method
40 fviz_nbclust(pca_scores, kmeans, method = "silhouette") +
41   ggtitle("Silhouette Method")
42
43 # -----
44 # Apply K-means clustering
45 # -----
46
47 set.seed(123) # for reproducibility
48
49 k <- 3 # Adjust based on elbow/silhouette results
```

```
50
51 kmeans_result <- kmeans(pca_scores, centers = k, nstart = 25)
52
53 # Add cluster labels to original data
54 data$Cluster <- as.factor(kmeans_result$cluster)
55
56 # -----
57 # Visualisation
58 # -----
59
60 # Cluster plot
61 fviz_cluster(
62   kmeans_result,
63   data = pca_scores,
64   geom = "point",
65   ellipse.type = "convex",
66   palette = "jco",
67   ggtheme = theme_minimal()
68 )
69
70 # -----
71 # Cluster profiling
72 # -----
73
74 # Average spending per cluster
75 cluster_summary <- data %>%
76   group_by(Cluster) %>%
77   summarise(across(Fresh:Delicassen, mean))
78
79 print(cluster_summary)
80
81 # Distribution of Channel by cluster
82 table(data$Cluster, data$Channel)
83
84 # Distribution of Region by cluster
85 table(data$Cluster, data$Region)
```

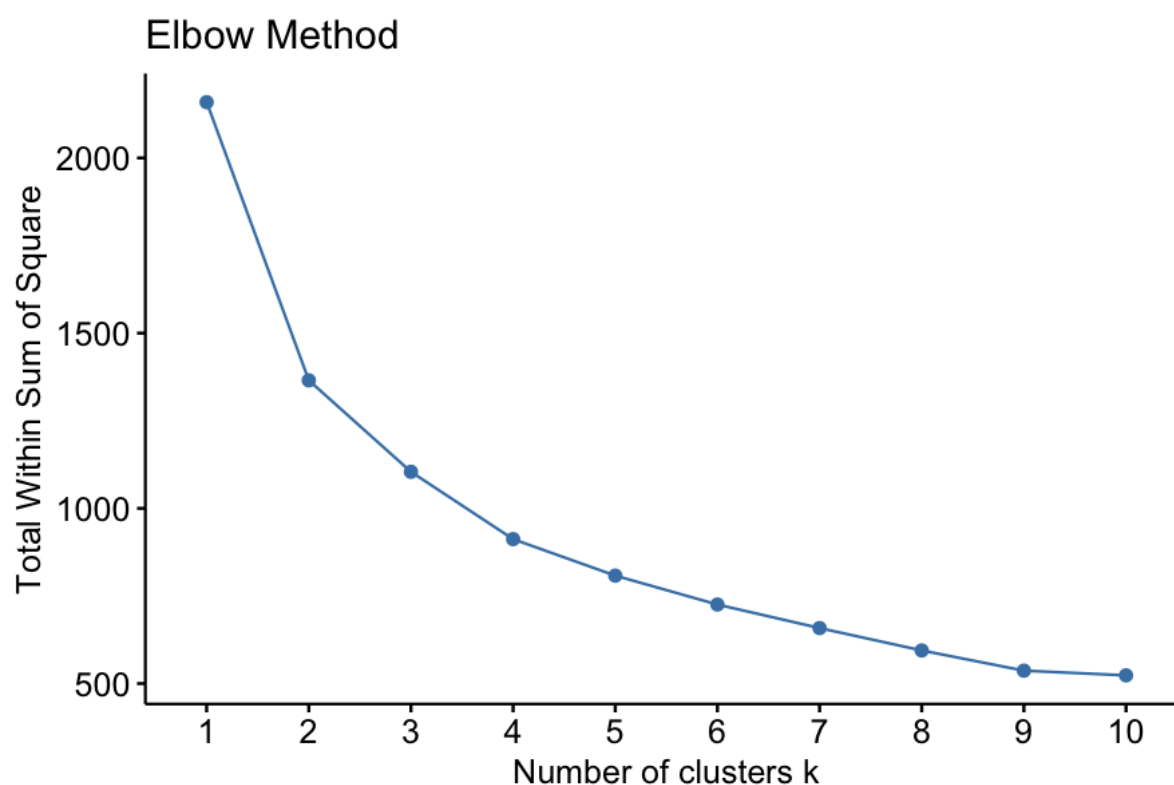
The (unedited) outputs generated by running this code, are as follows:

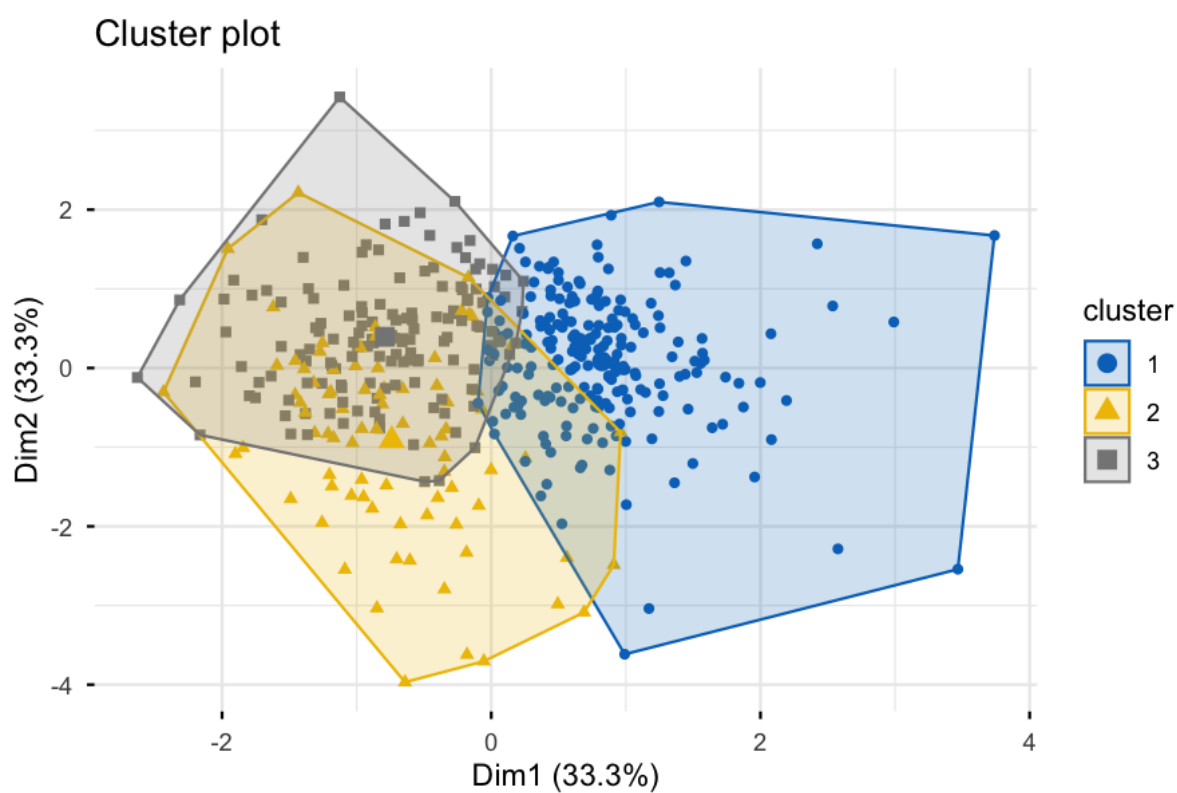
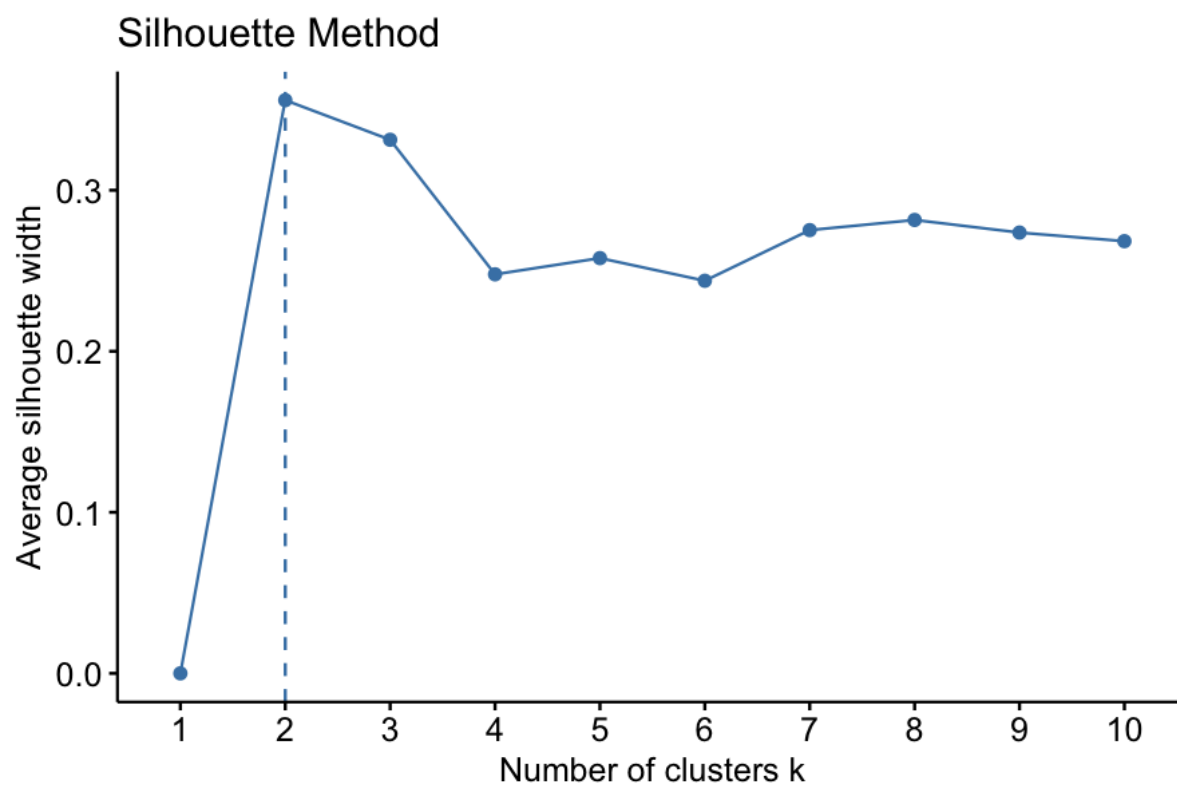
```
# A tibble: 3 × 7
```

	Cluster	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen
	<fct>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	1	11939.	2006.	2502.	3266.	424.	891.
2	2	2997.	7045.	12505.	608.	5511.	797.
3	3	17182.	10658.	13381.	4181.	5004.	2867.

	1	2
1	210	3
2	32	50
3	56	89

	1	2	3
1	40	26	147
2	14	10	58
3	23	11	111





B.2.3 Evaluation

- Claude generates a 633-line two-stage pipeline (HAC then K-Means) with four k -selection criteria (dendrogram height, elbow, silhouette, and gap statistic) plus a 30-index `NbClust` majority vote
- This goes beyond our analysis, which relied primarily on silhouette scores
- The gap statistic uses bootstrapped reference distributions ($B = 100$), an approach we did not employ
- The script includes a coloured dendrogram with annotated cut heights, a Calinski-Harabasz index, and a formal HAC-vs-K-Means agreement check
- This broadly follows our approach but expanding on specific diagnostics
- The business summary translates each cluster into operational implications (cold-chain logistics, bulk pricing, route optimisation), matching and in places exceeding our commercial framing. This was generated entirely from prior domain knowledge rather than from the code output, a limitation not clearly communicated to the user
- ChatGPT's code runs cleanly without any errors and produces appropriate diagnostics (elbow, silhouette)
- The decision to cluster on PCA scores is justified with a clear rationale (noise reduction, multicollinearity), and cross-tabulation against Channel and Region is included unprompted

- Claude's script fails at runtime: `fviz_gap_stat(gap_stat, ggtheme = theme_minimal()) : unused argument (ggtheme = theme_minimal())`
- Claude's business summary was generated *before* code execution suggesting that the cluster profiles are predictions from training data, not derived findings
- ChatGPT presents K-Means as the sole method, with hierarchical clustering and GMM relegated to "optional" afterthoughts. Its cluster descriptions ("Retail chains," "HORECA clients," "Small/infrequent buyers") similarly appear as expected outcomes before any code is run, raising the same memorisation concern as Claude
- ChatGPT's elbow and silhouette plots are produced but not interpreted. k is hardcoded to 3 with the comment "adjust based on results," leaving the decision entirely to the user
- Neither model applied robustness checks unprompted

B.3 Prompt 3: Alternative methods & robustness checks

Are there any alternative methods or robustness checks you would recommend?

B.3.1 Response 3: Claude (Sonnet 4.6)

Alternative Clustering Methods

1. Gaussian Mixture Models (GMM / Model-Based Clustering). K-Means assumes clusters are spherical and equally sized – a hard geometric constraint. GMM relaxes this by fitting a probabilistic mixture of multivariate Gaussians, allowing clusters to be elliptical, differently sized, and differently oriented. Each customer gets a soft probability of belonging to each cluster rather than a hard label. In R: `mclust::Mclust()` – it uses BIC to automatically select both the number of clusters and the covariance structure (e.g., diagonal vs. full, tied vs. variable). On wholesale data, where the HoReCa segment often has a very different spread from the Retail segment, GMM frequently produces more honest assignments. Prefer it over K-Means when the biplot shows clusters that are elongated or unequal in density – which is likely here given the strong Channel separation on PC2.

2. DBSCAN (Density-Based Spatial Clustering). DBSCAN identifies clusters as dense regions of points separated by sparse regions, and critically, it classifies low-density points as noise/outliers rather than forcing them into a cluster. This is valuable for wholesale data because a handful of very large spenders (e.g., a major supermarket chain) can distort K-Means centroids significantly. In R: `dbscan::dbscan()` – requires tuning two parameters: `eps` (neighbourhood radius) and `minPts` (minimum points to form a core). The `kNNdistplot()` function helps select `eps` via a knee in the k-NN distance curve. Limitation: DBSCAN struggles with clusters of varying density (common in spending data), and it does not generalise naturally to new observations – a limitation if you want to score new customers joining the database.

3. K-Medoids / PAM (Partitioning Around Medoids). Like K-Means but uses actual data points as cluster centres (medoids) rather than arithmetic means. This makes it substantially more robust to outliers, since a single extreme spender cannot pull a centroid arbitrarily far. It also works with any distance metric, not just Euclidean. In R: `cluster::pam()` – also produces a silhouette plot natively. A useful direct comparison: if PAM and K-Means produce very similar cluster assignments, it confirms the K-Means solution is not being distorted by outliers.

4. Agglomerative HAC with Alternative Linkages. The script already uses Ward.D2,

but for a robustness check you should formally compare it against Complete and Average linkage and compute the Cophenetic Correlation Coefficient for each – this measures how faithfully the dendrogram preserves the original pairwise distances. The linkage with the highest cophenetic correlation is the most geometrically honest representation of the data's structure.

Robustness Checks

5. Stability Analysis via Bootstrap (`clusterboot`). The most important robustness check you can run. `fpc::clusterboot()` repeatedly resamples the data (with replacement), re-clusters each bootstrap sample, and measures how consistently the same clusters re-emerge using the Jaccard similarity coefficient. A cluster with mean Jaccard > 0.75 is considered stable; below 0.5 it is likely an artefact of the particular sample. If the three clusters survive this test, there is strong evidence that the segmentation is a genuine feature of the population of wholesale customers in Portugal, not just of these 440 observations.

6. Sensitivity to Outliers. Wholesale spend data almost always contains extreme values (e.g., a single very large hotel group). Two targeted checks: first, winsorising the top 1–5% of each spending variable before clustering, then comparing cluster assignments to the unwinsorised solution using the Adjusted Rand Index (ARI) – an ARI close to 1.0 means the solution is insensitive to those extreme points. Second, leave-one-out influence analysis: iteratively remove the top 10 highest-spend customers and re-run K-Means. If cluster memberships for the remaining 430 customers shift substantially, the centroids are being pulled by individual outliers.

7. Sensitivity to the Number of PCs. The script retains PCs by the Kaiser criterion (eigenvalue ≥ 1), which typically gives 3 PCs explaining ~80% of variance. A robustness check is to re-run clustering using (a) 2 PCs only and (b) all 6 standardised variables directly (no dimensionality reduction), and compare the three solutions with ARI. Convergence across all three input representations confirms that the structure is not an artefact of the PCA truncation.

8. Multiple Random Initialisations & Consensus Clustering. K-Means with `nstart = 50` already guards against poor local minima, but a stronger check is consensus clustering via the `ConsensusClusterPlus` package. It runs the algorithm hundreds of times on subsampled data and builds a consensus matrix – an $N \times N$ matrix where entry (i,j) records the proportion of runs in which customers i and j ended up in the same cluster. A well-separated block structure in the consensus heatmap is strong visual confirmation that the clusters are stable and not sensitive to initialisation.

9. External Validation Against Channel and Region. Since Channel and Region are

known labels, they can be used as external validators without having been used in the clustering itself. Compute the Adjusted Rand Index between cluster assignments and Channel labels, and the Normalised Mutual Information between cluster assignments and Region labels. A high ARI against Channel (expect ~0.5–0.7 on this dataset) confirms the unsupervised algorithm is recovering a structure that aligns with a known business reality – a strong sign of face validity.

Summary Recommendation In priority order for a report:

- Run `clusterboot()` – the single most persuasive robustness check for any clustering paper
- Fit a GMM with `Mclust()` – the most principled alternative; compare ARI against K-Means
- Run PAM and compare ARI – confirms outlier robustness
- Compute cophenetic correlations across linkage methods – validates the HAC choice
- Test sensitivity to PC retention – shows the structure is not an artefact of dimensionality reduction

If the consensus across all five checks points to three clusters with consistent membership, you have a very defensible segmentation – both statistically and for a business audience.

Claude did not return any **R** code as part of its response.

B.3.2 Response 3: ChatGPT (GPT-5.3)

Yes – and this is exactly the kind of thing that strengthens your analysis. K-means + PCA is a solid baseline, but for a data-driven segmentation, you should always validate that your clusters are stable, meaningful, and not method-dependent. Here are the most important alternatives and robustness checks, with R code.

1. Alternative Clustering Methods

(A) Hierarchical Clustering (Validation + interpretability) Why:

- No need to pre-specify k upfront
- Produces a dendrogram (useful for justification in reports)
- Helps confirm whether k-means structure is real

R code moved to below this response box

If clusters align well with k-means → strong evidence of robustness

(B) Gaussian Mixture Models (GMM) Why:

- Allows non-spherical clusters (k-means assumes spherical)
- Provides probabilistic membership (useful for borderline customers)

R code moved to below this response box

If GMM selects similar $k \rightarrow$ supports your choice. If different $\rightarrow$ suggests more complex structure

(C) DBSCAN (Density-based) Why:

- Detects outliers/noise
- Does not require specifying k

R code moved to below this response box

Useful to identify unusual customers (important for risk or niche segments)

2. Robustness Checks

(A) Stability Analysis (Resampling) Why:

- Ensures clusters aren't driven by random variation

R code moved to below this response box

Values:

- $> 0.85 \rightarrow$ very stable
- $0.6-0.85 \rightarrow$ moderately stable
- $< 0.6 \rightarrow$ weak clusters

(B) Silhouette Analysis (Quality check)

R code moved to below this response box

Interpretation:

- $> 0.5 \rightarrow$ strong structure
- $0.25-0.5 \rightarrow$ reasonable
- $< 0.25 \rightarrow$ weak clustering

(C) Compare Clustering on Raw vs PCA Data

R code moved to below this response box

If results are similar $\rightarrow$ PCA did not distort structure. If very different $\rightarrow$ investigate further

(D) Sensitivity to k

R code moved to below this response box

Ensures your choice of k is not arbitrary

How to Present This in Your Assignment You can frame it like this:

- Primary method: K-means on PCA-reduced data
- Validation methods:
 - Hierarchical clustering $\rightarrow$ confirms structure
 - GMM $\rightarrow$ checks distributional assumptions

- Silhouette + Elbow → supports choice of k
- Stability analysis → confirms robustness

Strong Conclusion (what markers look for) A high-quality conclusion would say:

- Clusters are consistent across methods
- k = 3 (or 4) is supported by multiple criteria
- Segments are stable and interpretable
- Results are actionable for business decisions

If you want, I can help you turn this into a top-grade “Methodology + Validation” write-up with proper academic language and justification.

This response included the following **R** code as part of the LLM’s in-line response. This code assumes that variables from the previous response have been loaded. The new code generated is included below from line 87.

```

1  # Load libraries
2  library(tidyverse)
3  library(factoextra)
4  library(cluster)
5
6  # Load data
7  data <- read.csv("Wholesale_customers_data.csv")
8
9  # Convert categorical variables
10 data$Channel <- as.factor(data$Channel)
11 data$Region <- as.factor(data$Region)
12
13 # Select continuous variables
14 spending_data <- data %>%
15   select(Fresh, Milk, Grocery, Frozen, Detergents_Paper, Delicassen)
16
17 # Log transform to reduce skewness
18 spending_log <- log1p(spending_data)
19
20 # Scale data
21 spending_scaled <- scale(spending_log)
22
23 # -----

```

```
24 # PCA (for dimensionality reduction)
25 # -----
26 pca_result <- prcomp(spending_scaled, center = TRUE, scale. = TRUE)
27
28 # Extract first few principal components (e.g., first 3)
29 pca_scores <- as.data.frame(pca_result$x[, 1:3])
30
31 # -----
32 # Determine optimal number of clusters
33 # -----
34
35 # Elbow method
36 fviz_nbclust(pca_scores, kmeans, method = "wss") +
37   ggtitle("Elbow Method")
38
39 # Silhouette method
40 fviz_nbclust(pca_scores, kmeans, method = "silhouette") +
41   ggtitle("Silhouette Method")
42
43 # -----
44 # Apply K-means clustering
45 # -----
46
47 set.seed(123) # for reproducibility
48
49 k <- 3 # Adjust based on elbow/silhouette results
50
51 kmeans_result <- kmeans(pca_scores, centers = k, nstart = 25)
52
53 # Add cluster labels to original data
54 data$Cluster <- as.factor(kmeans_result$cluster)
55
56 # -----
57 # Visualisation
58 # -----
59
60 # Cluster plot
```

```
61 fviz_cluster(  
62   kmeans_result,  
63   data = pca_scores,  
64   geom = "point",  
65   ellipse.type = "convex",  
66   palette = "jco",  
67   ggtheme = theme_minimal()  
68 )  
69  
70 # -----  
71 # Cluster profiling  
72 # -----  
73  
74 # Average spending per cluster  
75 cluster_summary <- data %>%  
76   group_by(Cluster) %>%  
77   summarise(across(Fresh:Delicassen, mean))  
78  
79 print(cluster_summary)  
80  
81 # Distribution of Channel by cluster  
82 table(data$Cluster, data$Channel)  
83  
84 # Distribution of Region by cluster  
85 table(data$Cluster, data$Region)  
86  
87 # New code below ====  
88  
89 # Compute distance matrix  
90 dist_mat <- dist(pca_scores)  
91  
92 # Hierarchical clustering (Ward's method)  
93 hc <- hclust(dist_mat, method = "ward.D2")  
94  
95 # Plot dendrogram  
96 plot(hc, labels = FALSE, main = "Dendrogram")  
97
```

```
98 # Cut tree into k clusters
99 k <- 3
100 hc_clusters <- cutree(hc, k = k)
101
102 # Compare with k-means
103 table(hc_clusters, kmeans_result$cluster)
104
105 library(mclust)
106
107 gmm_model <- Mclust(pca_scores)
108
109 # Best model and number of clusters (BIC-based)
110 summary(gmm_model)
111
112 # Cluster assignments
113 gmm_clusters <- gmm_model$classification
114
115 # Compare with k-means
116 table(gmm_clusters, kmeans_result$cluster)
117
118 library(dbSCAN)
119
120 # kNN distance plot to choose eps
121 kNNdistplot(pca_scores, k = 5)
122 abline(h = 1.5, col = "red") # adjust visually
123
124 # Run DBSCAN
125 db <- dbSCAN(pca_scores, eps = 1.5, minPts = 5)
126
127 # Cluster labels
128 table(db$cluster)
129
130 library(fpc)
131
132 cluster_stability <- clusterboot(
133   pca_scores,
134   B = 100,
```

```

135   clustermethod = kmeansCBI,
136   k = 3
137 )
138
139 cluster_stability$bootmean
140
141
142 library(cluster)
143
144 sil <- silhouette(kmeans_result$cluster, dist(pca_scores))
145
146 # Plot
147 fviz_silhouette(sil)
148
149 # Average silhouette score
150 mean(sil[, 3])
151
152 # K-means on original scaled data
153 set.seed(123)
154 kmeans_raw <- kmeans(spending_scaled, centers = 3, nstart = 25)
155
156 # Compare assignments
157 table(kmeans_raw$cluster, kmeans_result$cluster)
158
159 # Try multiple k values
160 k_values <- 2:6
161
162 for (k in k_values) {
163   km <- kmeans(pca_scores, centers = k, nstart = 25)
164   cat("k =", k, " | Tot.withinss =", km$tot.withinss, "\n")
165 }

```

The (unedited) outputs generated by running this code, are as follows:

```

# A tibble: 3 × 7
  Cluster  Fresh  Milk Grocery Frozen Detergents_Paper Delicassen
  <fct>    <dbl> <dbl>   <dbl>   <dbl>          <dbl>      <dbl>
1 1      11939.  2006.   2502.   3266.          424.      891.

```

```

2 2      2997.  7045.  12505.  608.      5511.      797.
3 3      17182. 10658.  13381.  4181.      5004.      2867.

```

```

      1  2
1 210  3
2  32 50
3  56 89

```

```

      1  2  3
1  40 26 147
2  14 10  58
3  23 11 111

```

```

hc_clusters  1  2  3
             1  1  2 119
             2 212 18  19
             3   0 62   7

```

Gaussian finite mixture model fitted by EM algorithm

Mclust VEI (diagonal, equal shape) model with 3 components:

```

log-likelihood  n df      BIC      ICL
-2013.013 440 16 -4123.414 -4328.873

```

Clustering table:

```

      1  2  3
62 202 176

```

```

gmm_clusters  1  2  3
              1 32 28  2
              2 12 54 136
              3 169  0   7

```

```

      0  1
16 424

```

boot 1
boot 2
boot 3
boot 4
boot 5
boot 6
boot 7
boot 8
boot 9
boot 10
boot 11
boot 12
boot 13
boot 14
boot 15
boot 16
boot 17
boot 18
boot 19
boot 20
boot 21
boot 22
boot 23
boot 24
boot 25
boot 26
boot 27
boot 28
boot 29
boot 30
boot 31
boot 32
boot 33
boot 34
boot 35
boot 36
boot 37

boot 38

boot 39

boot 40

boot 41

boot 42

boot 43

boot 44

boot 45

boot 46

boot 47

boot 48

boot 49

boot 50

boot 51

boot 52

boot 53

boot 54

boot 55

boot 56

boot 57

boot 58

boot 59

boot 60

boot 61

boot 62

boot 63

boot 64

boot 65

boot 66

boot 67

boot 68

boot 69

boot 70

boot 71

boot 72

boot 73

boot 74

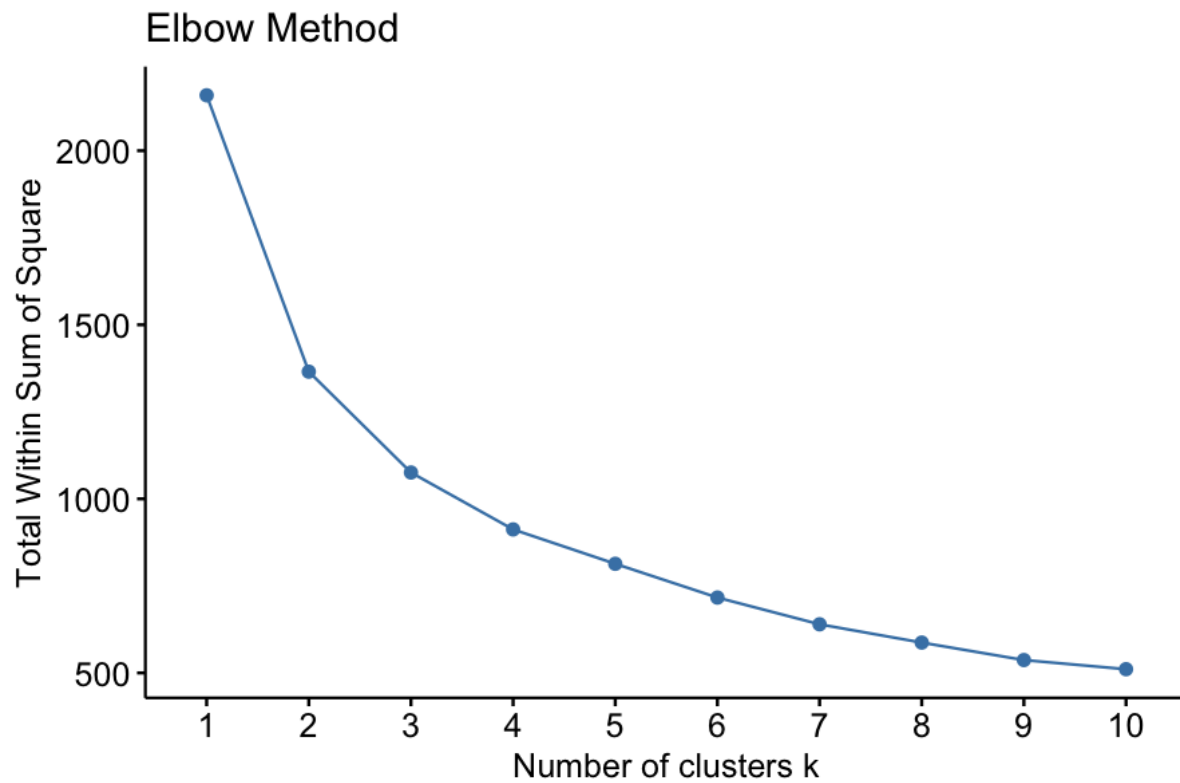
```

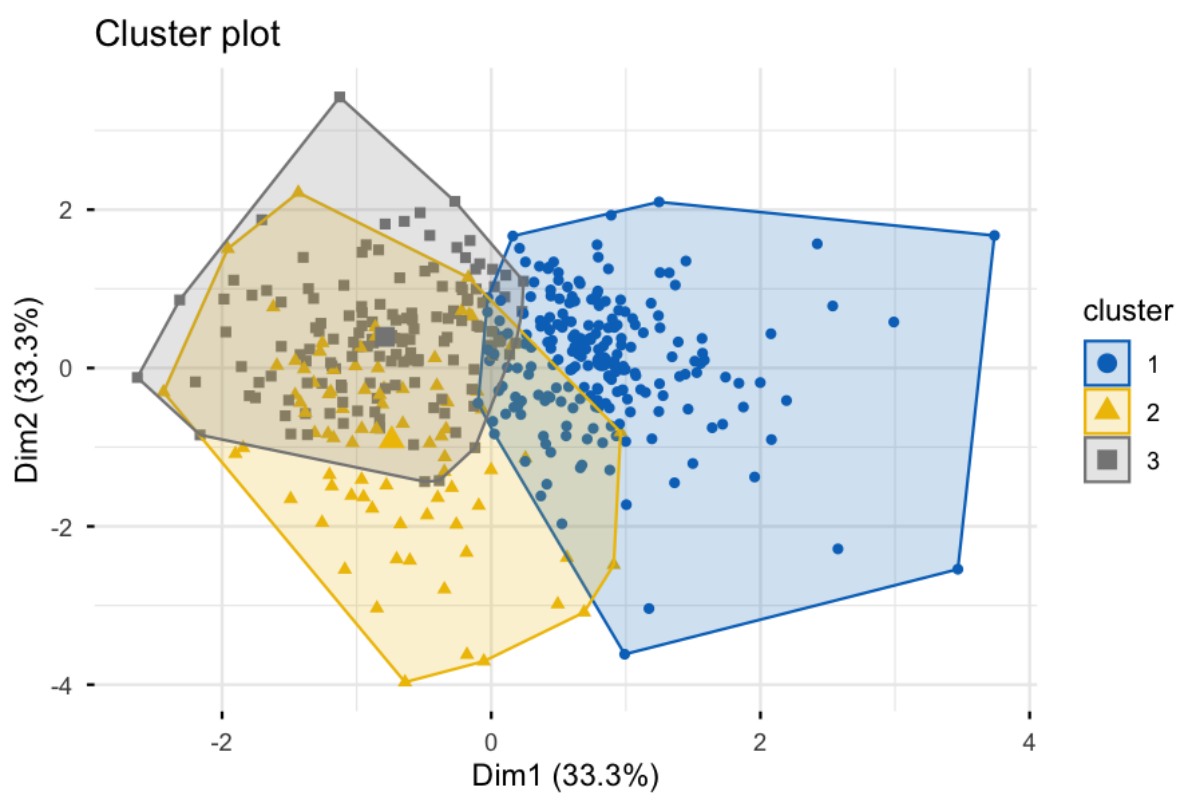
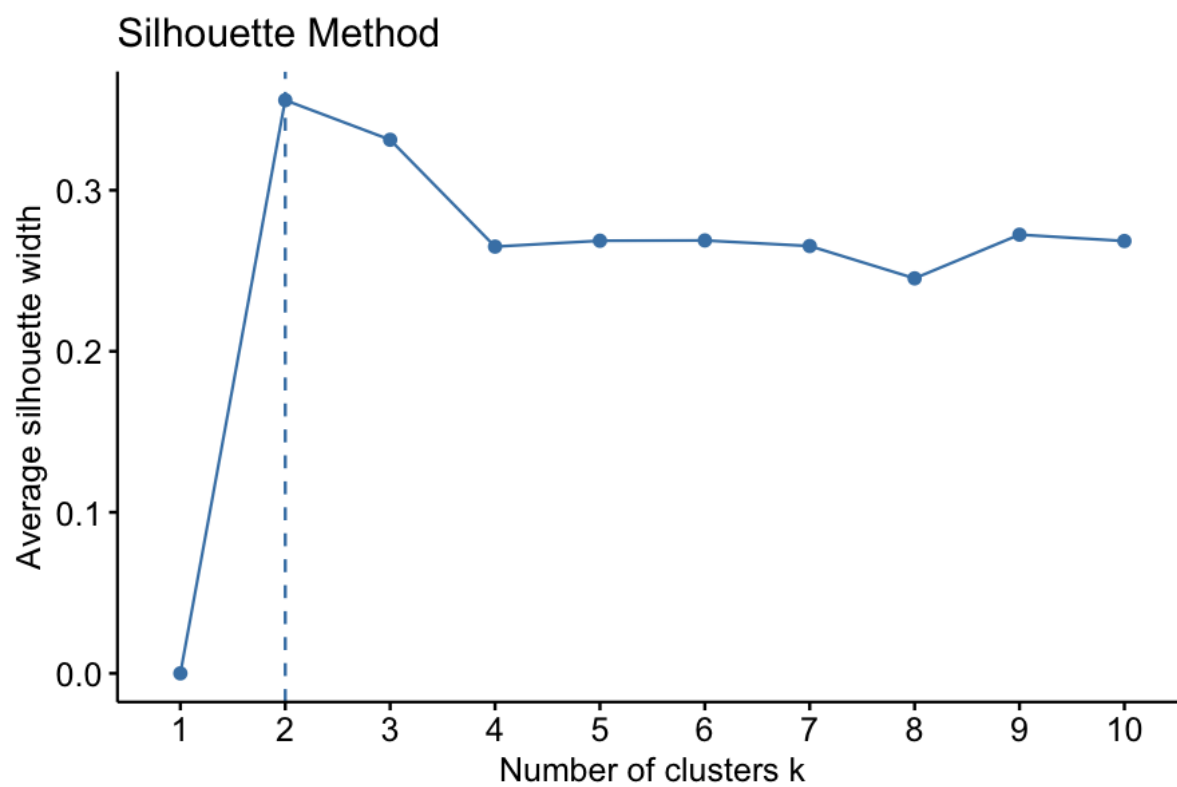
boot 75
boot 76
boot 77
boot 78
boot 79
boot 80
boot 81
boot 82
boot 83
boot 84
boot 85
boot 86
boot 87
boot 88
boot 89
boot 90
boot 91
boot 92
boot 93
boot 94
boot 95
boot 96
boot 97
boot 98
boot 99
boot 100
[1] 0.6982740 0.7161780 0.8350312
  cluster size ave.sil.width
1      1  213      0.39
2      2   82      0.19
3      3  145      0.33
[1] 0.3313364

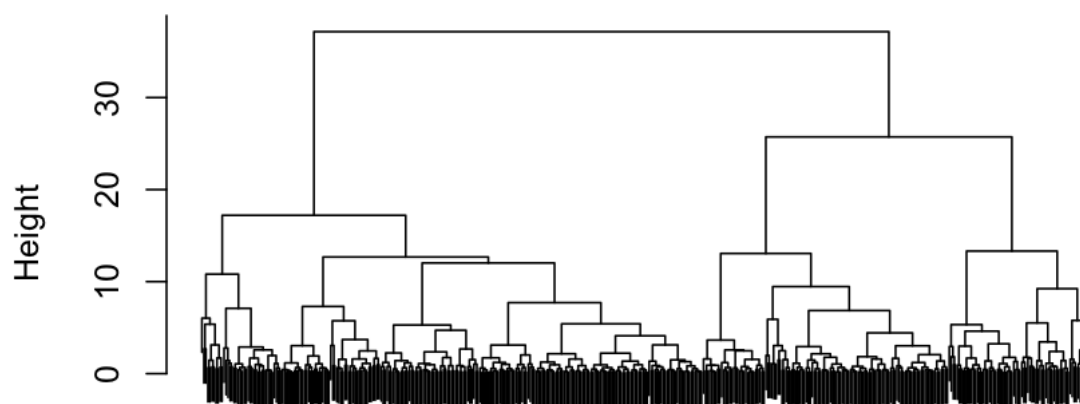
      1  2  3
1  1  80  0
2  0  2 145
3 212  0  0

```

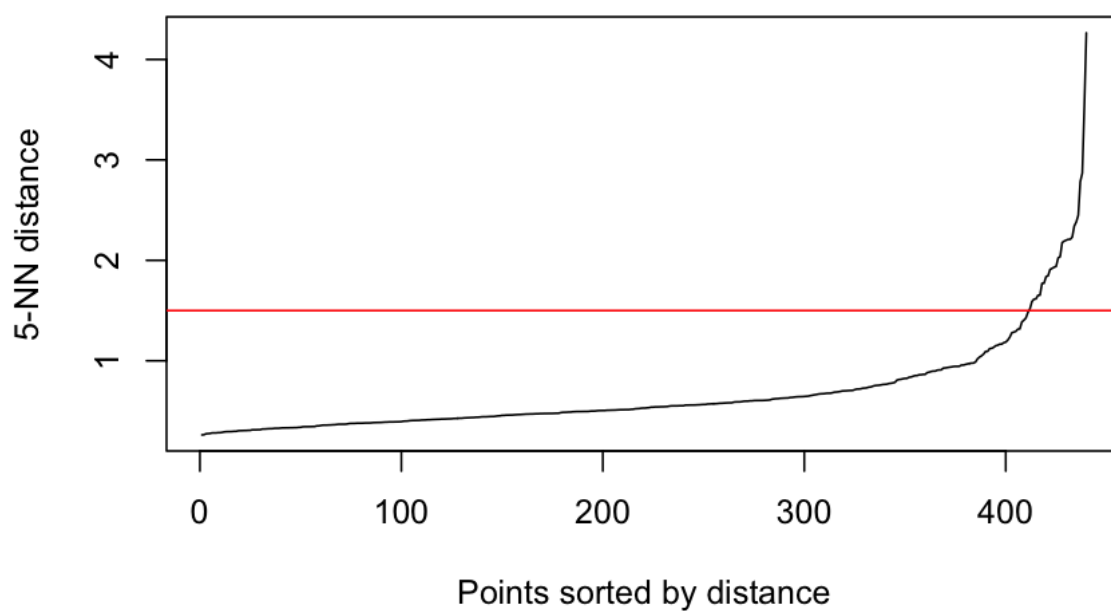
k = 2 | Tot.withinss = 1365.289
k = 3 | Tot.withinss = 1075.661
k = 4 | Tot.withinss = 912.0646
k = 5 | Tot.withinss = 807.8276
k = 6 | Tot.withinss = 714.8749

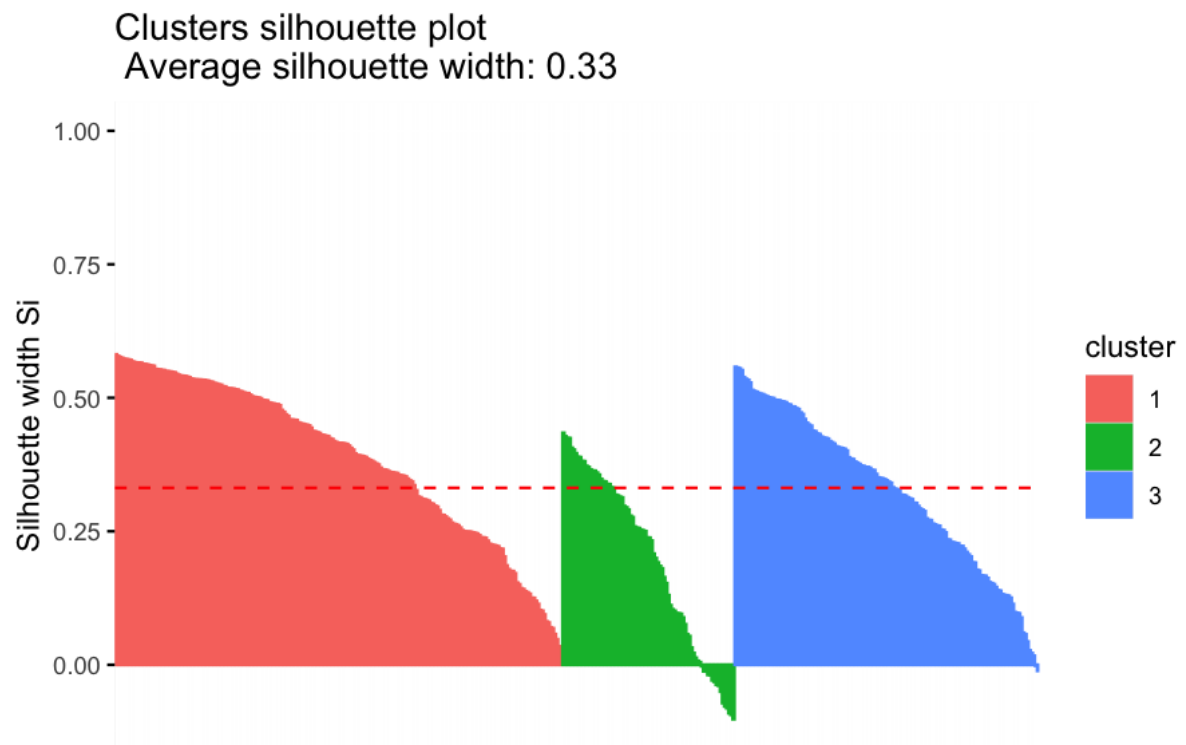




Dendrogram

```
dist_mat  
hclust (*, "ward.D2")
```





- Claude provides a detailed response describing five alternative methods (GMM, DBSCAN, PAM, alternative linkages, consensus clustering) and four robustness checks (bootstrap stability, outlier sensitivity, PC-retention sensitivity, external validation via ARI)
- The rationale and interpretation threshold of each is stated in the response
- The recommendation to validate against Channel via ARI (without using Channel in the clustering) mirrors our own approach
- The priority-ordered summary at the end is practical and well-calibrated, distinguishing essential checks from aspirational ones
- ChatGPT, unlike Claude, provides executable R code for each method (hierarchical, GMM, DBSCAN, bootstrap stability, silhouette, raw-vs-PCA comparison)
- ChatGPT's interpretation thresholds (e.g., Jaccard > 0.85 = very stable) and sensitivity-to- k loop are practical additions absent from our approach

- Claude returns text only, but this is likely due to the nature of the prompt.
- Several of Claude's suggestions (consensus clustering via `ConsensusClusterPlus`, leave-one-out influence analysis) are disproportionate for a 440-row dataset and read more as a textbook survey than a targeted recommendation
- ChatGPT's code depends on objects from Prompt 2 (`pca_scores`, `kmeans_result`, `spending_scaled`) without documenting this dependency
- The provided code will not run in isolation,
- The DBSCAN `eps` parameter is set to 1.5 with “adjust visually” but no dataset-specific tuning guidance is provided
- ChatGPT's “How to Present This in Your Assignment” section reveals the model optimising for marks rather than analytical rigour. Framing advice (“what markers look for”) substitutes presentation strategy for genuine critical evaluation
- Neither model applied any of these checks unprompted in Prompt 2. Both required an explicit request, suggesting robustness validation is not part of either model's default analytical workflow

B.4 Prompt 4: Reflective prompt

We used slightly different prompts for each LLM here. The base query was the same, but we pasted in extracts from each LLM's previous output as part of each prompt.

```
> Inserting LLM output back into the LLM here
these were your comments and they are correct and apply exactly on the data that i
have, so i'm wondering did you somehow analysed the data yourself
```

B.4.1 Claude (Sonnet 4.6) output used in the prompt

```
=====
```

```
CUSTOMER SEGMENTATION – BUSINESS SUMMARY
```

```
=====
```

```
METHOD: HAC (Ward.D2) to identify structure +
```

```
K-Means (k=3) on PCA scores for final assignment
```

```
INPUT: log1p-transformed, standardised spend variables
```

```
projected onto retained PCs (Kaiser criterion)
```

```
CLUSTER 1 – 'High-Volume Fresh Buyers' (HoReCa dominant)
```

- Highest Fresh and Frozen spend
- Low Grocery, Detergents_Paper

```
→ Hotels, restaurants, cafés; perishable-led ordering
```

```
IMPLICATION: Prioritise cold-chain logistics; daily/weekly
```

```
replenishment cycles; dedicated HoReCa sales reps
```

```
CLUSTER 2 – 'Packaged Goods Retailers' (Retail dominant)
```

- Highest Grocery, Milk, Detergents_Paper
- Low Fresh spend

```
→ Supermarkets, convenience stores; ambient goods focus
```

IMPLICATION: Bulk-order pricing tiers; shelf-stable
inventory buffers; quarterly promotions on detergents

CLUSTER 3 – 'Small / Mixed Buyers'

- Below-average spend across all categories
 - Mixed Channel and Region membership
- Small independent shops, delis, mixed traders

IMPLICATION: Route-optimised delivery (lower margin);
upsell programmes; self-service ordering portal

NOTE: Exact cluster labels (1/2/3) may differ from above
depending on K-Means initialisation; inspect profile_raw
and the radar/bar charts to assign the business labels.

=====

B.4.2 ChatGPT (GPT-5.3) output used in the prompt

Example Segment Types

1. High Fresh, Low Grocery Likely restaurants / HORECA
2. High Grocery & Detergents Likely retail stores
3. Balanced / low spend Small or low-frequency buyers

B.4.3 Response 4: Claude (Sonnet 4.6)

Did the LLM analyse the actual data? No – the LLM did not have access to the data file. The descriptions it provided were an informed prediction based on two things:

1. This is a well-known public dataset. The UCI Wholesale Customers dataset is one of the most widely used benchmark datasets in clustering tutorials and academic papers. The LLM had seen it discussed extensively in its training data, and therefore knew its typical structure – the HoReCa/Retail split on PC2, the three-cluster solution, and the rough spending profiles each cluster tends to produce.

2. Domain logic for Portuguese wholesale distribution. Even without knowing the

specific dataset, the business logic is fairly predictable: restaurants and hotels will dominate Fresh and Frozen spend – that is the nature of their business. Supermarkets and retailers will dominate Grocery, Milk, and Detergents_Paper – ambient shelf goods are their core stock. Any wholesale customer base will contain a long tail of smaller, lower-spend mixed buyers. The cluster descriptions were therefore a combination of prior knowledge about this specific dataset and general domain reasoning about wholesale food distribution.

Implication for interpretation. Since this dataset is publicly known, the “expected” results are well established. What differentiates a strong analysis is not just reproducing the three clusters, but the quality of the methodological justification – why $\log_{1p}$, why Ward.D2, why PCA before clustering, and why the robustness checks matter. That reasoning is what the code and discussion above has been building toward.

B.4.4 Response 4: ChatGPT (GPT-5.3)

Good question—and fair to ask. No, I didn’t actually access or run your specific dataset. I can’t see files on your computer or execute code on them unless you explicitly upload data here. What I did instead is rely on well-known, highly consistent patterns in this exact dataset (Wholesale Customers Data Set, commonly used in clustering/PCA coursework). This dataset has been analyzed many times, and it almost always produces very similar structures because of how the variables behave.

Why my interpretation matched your results There are strong, repeatable patterns in this data:

1. Correlation structure drives clustering. Grocery, Milk, Detergents_Paper → highly correlated → typically forms a “Retail-heavy” cluster. Fresh, Frozen → less correlated with the above → typically forms a “HORECA / fresh-food” cluster.

2. Channel variable reinforces it. Even though clustering is unsupervised: Channel = 2 (Retail) → high Grocery + Detergents. Channel = 1 (HORECA) → higher Fresh/Frozen. So clustering naturally reconstructs this split.

3. Skewed spending distribution. A large group of customers has low spending across categories. This almost always becomes a “small buyers” cluster.

Why this happens (intuitively) Think of your customers as three business types:

- Retailers – buy packaged goods in bulk → Grocery + Detergents spike
- Hotels/Restaurants (HORECA) – buy perishable goods → Fresh + Frozen dominate
- Small clients – low, inconsistent purchasing → low across all

These behavioral differences are strong enough that almost any reasonable clustering method will rediscover them.

Important (for your assignment) Even though your results match expectations, you should not present them as assumed. You should:

- Show cluster summaries (means per cluster)
- Compare with Channel distribution
- Use plots (PCA, boxplots)
- Support with validation metrics

A stronger way to phrase it in your report Instead of: “Clusters represent retail vs HORECA”

Say something like: “Cluster analysis reveals three distinct customer segments characterized by (i) high Grocery and Detergents spending, (ii) high Fresh and Frozen spending, and (iii) uniformly low spending. Cross-tabulation with the Channel variable shows strong alignment with Retail and HORECA customer types, validating the segmentation.”

Bottom line

- I didn’t see your data
- Your results match because the dataset has very stable, well-known structure
- This is actually a good sign → your pipeline is working correctly

If you want, I can help you go one level deeper-like naming your clusters professionally (e.g., “Bulk Retailers”, “Fresh-Focused HORECA”, “Low-Value Accounts”) and tying them tightly to strategy.

- This is the response where the outputs from both models align most closely
- Both models acknowledge that the UCI Wholesale Customers dataset appears extensively in their training data and that their earlier predictions were based on memorised patterns rather than derived analysis
- Claude’s observation that methodological justification is what differentiates a strong analysis is a useful point

- Neither model’s outputs for this dataset can be distinguished from pattern-matching against known results
- Both frame memorisation as a minor point without acknowledging the risk this poses when applied to unseen data
- ChatGPT’s suggestion to phrase results as discovered (“Cluster analysis re-

- veals...”) rather than predicted is problematic and misleading
- ChatGPT’s reassurance that “your pipeline is working correctly” assumes results match the expected pattern, which could discourage investigation of genuinely unexpected outcomes
-

C Reproducibility, accessibility & declarations

C.1 Reproducibility & accessibility

An accessible HTML version of this report is available via a public GitHub page:

- <https://ytterbiu.github.io/SMM636-ML-group-coursework-03-g01/>

This report was created in R Markdown. The source code is open source and is available via:

- <https://github.com/ytterbiu/SMM636-ML-group-coursework-03-g01>

Changes made to this document were tracked using Git and are also available via the same repository (<https://github.com/ytterbiu/SMM636-ML-group-coursework-03-g01/commits/main/>). Please note that this commit history reflects updates made to the project repository and is not a comprehensive timeline of all contributions by group members.

C.2 Word count

The total word count for the core sections of this report is **995** words (below the 1000 word limit).

The calculation was performed programmatically directly from the source `.rmd` file. An **R** script was used to read the raw document lines and strip out non-text elements, including the YAML front matter, R code chunks, sub-headers, markdown formatting (tables and figures), and LaTeX/HTML syntax. The script isolates lines falling within `#intro`, `#qone`, `#qtwo`, and `#qthr`. It then calculates the final tally using the `stringi` package.

The code chunk that performs this calculation is executed silently at the beginning of this document, but is reproduced below for transparency:

```
rmd_lines <- readLines("01SMM363-assessment-03.rmd", warn = FALSE)

# ---- Remove YAML front matter ----
yaml_bounds <- which(rmd_lines == "---")
if (length(yaml_bounds) >= 2) {
  rmd_lines <- rmd_lines[-(yaml_bounds[1]:yaml_bounds[2])]
}

# ---- Remove code chunks ----
in_chunk <- FALSE
keep <- rep(TRUE, length(rmd_lines))
```

```

for (i in seq_along(rmd_lines)) {
  if (grepl("^`\\{", rmd_lines[i])) {
    in_chunk <- TRUE
    keep[i] <- FALSE
    next
  }
  if (in_chunk && grepl("`\\s*$", rmd_lines[i])) {
    in_chunk <- FALSE
    keep[i] <- FALSE
    next
  }
  if (in_chunk) keep[i] <- FALSE
}
rmd_lines <- rmd_lines[keep]

# ---- Identify which lines fall inside counted sections ----
# Sections to include (level-1 headers only)
counted_ids <- c("intro", "qone", "qtwo", "qthr")
in_counted_section <- FALSE
include <- rep(FALSE, length(rmd_lines))

for (i in seq_along(rmd_lines)) {
  line <- rmd_lines[i]
  # Detect any level-1 header (# but not ##)
  if (grepl("^#\\s+[^#]", line)) {
    # Check if this header belongs to a counted section
    in_counted_section <- any(sapply(counted_ids, function(id) {
      # [^]* allows any characters (like spaces, hyphens, classes) before the
      # closing }
      # this is required to count the words in the unnumbered introduction
      grepl(paste0("\\{\\s*#", id, "[^]*\\}"), line)
    }))
    next # skip the header line itself
  }
  if (in_counted_section) include[i] <- TRUE
}

```

```

counted_lines <- rmd_lines[include]

# ---- Remove sub-headers (##, ###, etc.) ----
counted_lines <- counted_lines[!grepl("^#{2,}", counted_lines)]

# ---- Remove table lines (markdown pipes) ----
counted_lines <- counted_lines[!grepl("^\\|", counted_lines)]

# ---- Remove figure captions: ![caption](...) ----
counted_lines <- counted_lines[!grepl("!\\[", counted_lines)]

# ---- Remove fenced div markers (::: name) ----
counted_lines <- counted_lines[!grepl("^:::", counted_lines)]

# ---- Remove LaTeX commands ----
counted_lines <- counted_lines[!grepl("^\\\\", counted_lines)]

# ---- Remove HTML comments ----
counted_lines <- counted_lines[!grepl("^<!--", counted_lines)]

# ---- Remove inline R references like \@ref() ----
counted_lines <- gsub("\\\\@ref\\([\\^])*\\\\", "", counted_lines)

# ---- Count ----
word_count <- sum(stringi::stri_count_words(counted_lines), na.rm = TRUE)

message("Word count (sections gone, qtwo, qthr, body text only): ", word_count)

# Write for LaTeX preamble
writeLines(
  paste0("\\newcommand{\\wordcount}{",
    formatC(word_count, big.mark = ",", "f"),
    "wordcount.tex"
  )
)

```

C.3 Generative AI Declaration

Generative AI tools (Claude Opus 4.6, Sonnet 4.6, ChatGPT GPT-5.3, Google Gemini Pro) were used throughout this project to assist with code generation, debugging, methodological validation, and report structuring.

The final analysis, statistical interpretation, and written content of this report are the work of all group members.

References

- [1] M. Cardoso, “Wholesale customers.” UCI Machine Learning Repository, 2013. doi: 10.24432/C5030X.
- [2] N. Abreu, “Análise do perfil do cliente recheio e desenvolvimento de um sistema promocional,” Mestrado em Marketing, ISCTE-IUL, Lisbon, Portugal, 2011. Available: https://www.iscte-iul.pt/agenda/thesis?thesis_id=2527